

IntentChecker: A Debugging Assistant for Validating Developer Intent to Mitigate Numeric Logic Errors in Solidity

Inseong Jeon¹, Sundeuk Kim¹, Hyunwoo Kim¹, Hoh Peter In^{1*}

^{1*}Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Smart contract development lacks the mature debugging infrastructure available in traditional software engineering. In particular, numeric logic errors—where program behavior diverges from developer intent—are difficult to address. Whether such errors are present depends on the developer’s intended numeric behavior, which is not expressed in the code itself. Existing detection tools, designed to operate from source code alone in a testing context, rely on pre-defined vulnerability patterns and offer no mechanism for developers to declare and validate numeric intent during development. Errors not covered by these patterns therefore have no practical way of early prevention. This paper presents IntentChecker, a development-time intent validation tool for Solidity that operates at multi-contract single-transaction scope and shifts the detection target from pre-defined patterns to developer-defined intent. We propose an intent model with two types: @During for intended relationships at specific program points within a function, and @Post for intended relationships after function execution. A validation engine then checks whether the intervals computed by abstract interpretation satisfy the declared intent and reports a validation result together with a risk score. We evaluate IntentChecker on real-world numeric logic errors and show that it provides a detection signal complementary to existing pattern-based tools, with analysis latency suitable for interactive development.

Keywords: Solidity, Numeric Logic Error, Debugging Assistant, Intent Model, Intent Validation

1 Introduction

The debugging toolchain available to smart contract developers remains far less mature than what traditional software engineering provides, and the immutability of deployed contracts further amplifies the cost of bugs that go undetected (Soud et al., 2024). This immaturity is particularly acute for *numeric logic errors*—bugs in which a function executes without raising any compile-time or runtime exception but nevertheless produces values that violate the developer’s intended numerical behavior. Because correctness hinges on what the developer intended for a given computation—information that the source code does not capture—compilers, type checkers, and generic static analyzers are structurally blind to this class of bugs. A recent empirical study (Chaliasos et al., 2024) confirms this in practice: practitioners identified logic errors as the most difficult vulnerability type to detect, and reported that existing security tools are least effective against them.

Several recent tools—most notably NumScout (Chen et al., 2025) and GPTScan (Sun et al., 2024)—explicitly target numeric logic errors. These tools face an **inherent difficulty of the detection paradigm** rather than a mere engineering gap. First, without access to the developer’s actual intent, any detector must fall back on a curated catalog of codified patterns (e.g., division-in-path, operator order, wrong checkpoint order). Second, because these catalogs are necessarily incomplete, tool coverage is orthogonal: error patterns targeted by GPTScan are missed by NumScout, and vice versa, and no single tool can be expected to unify them. These are not failures of any specific tool but structural consequences of attempting to detect intent-dependent bugs without an explicit representation of that intent.

Compounding the above difficulty is a second, **temporal** limitation. Both NumScout and GPTScan are designed to run over completed code in auditing or testing contexts, after the function under inspection is fully written. At edit time, when the developer’s intent is still fresh in mind and iteration cost is lowest, there is currently no tool that lets the developer declare “*I expect variable v to stay within range R at this point*” and immediately see whether that intent is consistent with the abstract execution of the function being written. This edit-time gap is the central problem this paper addresses.

Our starting point for closing this gap is SOLQDEBUG (Jeon et al., 2025), our previously published interactive debugger for Solidity. SOLQDEBUG provides the missing piece of infrastructure: an interactive parser and an incremental control-flow-graph builder that together drive interval-domain abstract interpretation at statement-level granularity, accepting developer-supplied range annotations (`@StateVar`, `@LocalVar`) to inject symbolic inputs. However, SOLQDEBUG only *observes* variable intervals; it does not validate any intended relationships between them, and its analysis scope is restricted to a single contract and a single transaction. SOLQDEBUG’s own conclusion explicitly leaves numerical defects detection for future work. This paper realizes that future work.

We present INTENTCHECKER, a development-time intent validation tool that builds on SOLQDEBUG’s interactive analysis infrastructure and adds three contributions. *First*, we introduce an *intent annotation model* consisting of two annotation

types that align with the stages of variable flow in a function body: **@During** annotations specify intended relationships that must hold at specific program points within a function, while **@Post** annotations specify intended relationships that must hold after the function completes execution. *Second*, we develop a *validation engine* that maps each annotation clause to an interval-domain comparison and produces a graded signal of *Satisfied*, *Violated*, or *Warning*, together with a risk score on a 0–10 scale and the sub-intervals in which the intent may be violated. The underlying comparison algorithm is exhaustively verified against brute-force ground truth across all possible topological configurations of two integer intervals. *Third*, we extend SOLQDEBUG’s analysis scope from single-contract to *multi-contract single-transaction*, adding support for inherited contracts, library delegatecalls, interface-typed variables, and a practically useful arithmetic subset of inline assembly, and we introduce a new **@IReturn** debug annotation that injects range bindings into interface call return values, preserving the same “unknown value, developer-specified range” semantics already used by SOLQDEBUG’s existing debug annotations.

To ground our motivation in concrete numbers, we analyzed 1,023 non-trivial functions from DAppSCAN-source (Zheng et al., 2024) (see Table 1) and found that numeric variable types appear in 82% of these functions, with `uint` alone used in 78%. Since numeric computations form the core of smart contract business logic—handling token transfers, fee calculations, and balance updates—explicitly specifying developer intent for these operations is both necessary and, as our evaluation shows, practical.

We evaluate INTENTCHECKER on 89 real-world numeric logic errors drawn from Web3Bugs (Zhang et al., 2023) and NumScout (Chen et al., 2025). The evaluation organizes itself around three research questions: the extent to which INTENTCHECKER can surface real-world numeric logic errors through its graded signal (RQ1), the annotation model’s expressibility ceiling for cases that require bug awareness (RQ2), and how INTENTCHECKER’s feedback compares with existing pattern-based detection tools in terms of signal characteristics, latency, and coverage complementarity (RQ3). Importantly, our positioning with respect to existing tools is complementary rather than competitive: INTENTCHECKER fills a distinct place in the development workflow by enabling intent-driven validation during editing, and is intended to coexist with auditing-time detectors rather than replace them.

Contributions.

- We identify an unfilled gap in current Solidity tooling: the absence of a development-time mechanism for declaring and validating numeric intent, and we show that this gap is not a minor engineering oversight but a structural consequence of how existing numeric logic detection tools are designed (§2).
- We present an intent annotation model in which **@During** and **@Post** annotations express intended numeric relationships at program-point and function-level granularities, together with a validation engine that produces a graded signal (*Satisfied* / *Warning* / *Violated*) with a risk score and violated region (§4).
- We extend SOLQDEBUG’s single-contract single-transaction analysis scope to a multi-contract single-transaction scope that handles inheritance, libraries via delegatecall, interfaces, and a subset of inline assembly, and we introduce the **@IReturn** debug annotation to bridge the remaining cross-deployment gap (§3).

Table 1: Distribution of Variable Types in Smart Contract Functions

Category	Number of Functions	Percentage (%)
Total Functions Analyzed	1524	
Excluded Functions	501	
Functions Considered for Analysis	1023	
Functions using <code>uint</code>	798	78.00587
Functions using <code>int</code>	5	0.48876
Functions using <code>uint</code> and <code>int</code>	41	4.10557
Functions using other types	178	17.39980

- We evaluate INTENTCHECKER on 89 real-world numeric logic errors and position it against NumScout and GPTScan along three axes—signal characteristics, per-case latency, and coverage complementarity—demonstrating that the three tools occupy distinct points in the development/auditing timeline rather than forming a single comparison along detection accuracy (§5).

The remainder of this paper is organized as follows. Section 2 motivates the development-time intent validation problem and surveys the structural limitations of existing detection tools. Section 3 recaps SOLQDEBUG and describes the scope extensions required for intent validation over multi-contract code. Section 4 presents the design of INTENTCHECKER, including its intent model and validation engine. Section 5 presents evaluation results across the three research questions. Section 6 discusses the annotation authoring burden and remaining technical limitations. Section 7 reviews related work, and Section 8 concludes.

2 Background

2.1 Numeric Logic Errors in Solidity

Recent research has identified various numeric-related weaknesses in smart contracts (Chen et al., 2025; Sun et al., 2024; Soud et al., 2024; Zhang et al., 2023; Zhou et al., 2023). Building on these findings, numeric logic errors can be defined as follows:

Definition 1 (Numeric Logic Error) Bugs that allow transactions to complete successfully without compile-time or runtime exceptions, but violate the developer’s intended numerical or state relationships during function execution.

These errors can manifest in different execution contexts. Single-transaction errors occur within the execution of a single function call—where computation, function calls, and output generation happen in one atomic sequence. In contrast, multiple-transaction errors only emerge through sequences of function calls, requiring analysis of inter-function dependencies and contract-level invariants. Examples of multiple-transaction errors include Risky First Deposit and Price Oracle Manipulation (Sun et al., 2024; Zhang et al., 2023), where attackers exploit state changes across multiple function invocations.

This paper focuses on single-transaction function execution, as this aligns with the development-phase validation model where developers can immediately verify individual function behaviors against their intentions. The representative single-transaction numeric logic error patterns identified in prior work are summarized below:

- **Div In Path** (Chen et al., 2025): Division results used in conditions can lead to unexpected control flow due to integer truncation.
- **Operator Order Issue** (Chen et al., 2025): Arithmetic order matters: $a/b*c \neq a*c/b$ due to intermediate truncation.
- **Exchange Problem** (Chen et al., 2025): Rounding errors in exchange calculations may result in zero output despite valid input, or receiving tokens without payment.
- **Precision Loss Trend** (Chen et al., 2025): Integer division’s inherent truncation tends to favor one side, potentially misaligning with protocol intent.
- **Minor Amount Retention** (Chen et al., 2025): Integer division in distributions leaves remainders unallocated to any recipient, causing permanent lock.
- **Erroneous Accounting** (Sun et al., 2024; Zhang et al., 2023): Incorrect implementation of business model formulas introduces errors that accumulate into substantial losses. Representative sub-patterns include:
 - *Wrong Interest Rate Order*: Balance calculations using outdated or prematurely updated interest rates yield incorrect results.
 - *Wrong Checkpoint Order*: Incorrect ordering between state updates and checkpoints leads to reward miscalculation.
- **Inconsistent State Updates** (Zhang et al., 2023): A required state variable update is omitted, causing subsequent calculations to use stale values. For example, failing to accrue interest before a balance query produces incorrect results.
- **Greedy Contract** (Soud et al., 2024): Contract can receive funds but lacks proper logic to withdraw them, causing permanent lock.

Despite their diversity, these error patterns share a common root: the actual behavior diverges from what the developer intended. A straightforward consequence is that any tool that hopes to flag such bugs reliably must reason about the intended behavior, not only the written code. The next subsection examines how existing tools handle—or rather, fail to handle—this requirement.

2.2 Limitations of Current Work

Several tools explicitly target numeric logic errors. NumScout (Chen et al., 2025) catalogs six numerical defect patterns (Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, Minor Amount Retention, and Profit Opportunity) and detects them through LLM-pruned symbolic execution. GPTScan (Sun et al., 2024) addresses a different group of logic vulnerabilities, including several accounting-related patterns, by combining GPT-based scenario matching with lightweight program analysis. Other works (Zhang, 2024; Wang et al., 2024b) take closely related approaches. These tools represent the current state of the art and have

meaningfully advanced the field. However, when viewed through the lens of the problem defined in §2.1, they share two structural limitations that are not engineering oversights but rather direct consequences of their design.

(1) Operation context: post-hoc auditing rather than development. Existing numeric logic error detection tools are designed to run over completed code, and are positioned within auditing or testing workflows. They accept a contract (or a set of contracts) and emit findings as a report. They do not run inside the developer’s editor as code is being written, nor do they accept developer-supplied specifications about the intended behavior of the function currently under the cursor. As a consequence, the developer’s intent is not part of the input at all. Any detector operating in this context must recover intent indirectly: either by hard-coding intent assumptions as “patterns” that describe what buggy code looks like (NumScout), or by asking a language model to infer intent from code, comments, and public documentation (GPTScan, SmartInv (Wang et al., 2024b)). Both strategies work only to the extent that the assumed intent coincides with the actual developer intent in the specific contract being analyzed.

(2) Approach: pattern-based coverage is orthogonal across tools. Because the operation context rules out direct access to intent, each tool must commit to a fixed catalog of patterns. These catalogs are necessarily incomplete: NumScout’s six numerical defect patterns and GPTScan’s accounting-related patterns target disjoint regions of the bug space. A bug matching a NumScout pattern is, in general, not matched by GPTScan, and vice versa. The problem is not that one catalog is better than the other; it is that neither catalog can be extended to cover all intent-dependent bugs, because intent itself is not expressible as a static pattern over code. This orthogonality means that “adding another pattern-based tool” cannot in principle converge on full coverage.

These two limitations are structural, not tool-specific. We emphasize that we do not characterize the above limitations as failures of NumScout or GPTScan. Both tools are well-engineered within the design space they chose. The point is that the design space itself—post-hoc pattern-based detection without explicit intent input—has an upper bound, and the field has largely reached it. To go further, one must either enlarge the input (ask the developer to supply their intent) or move the tool earlier in time (run it during editing, when the developer is still available to supply that intent). The approach taken in this paper is to do both.

2.3 Proposed Overview

The structural limitations above suggest a complementary direction rather than a competing one. If existing detectors cannot see the developer’s intent, a tool that sits in the editor and explicitly asks the developer to declare their intent can fill the remaining space without contradicting the existing tools’ strengths. Our prior work, SOLQDEBUG (Jeon et al., 2025), already provides much of the infrastructure needed for such a tool: an interactive parser, an incremental control-flow graph, and interval-domain abstract interpretation that accepts developer-supplied range annotations (@StateVar,

@LocalVar) and produces line-level feedback as code is being written. What SOLQDEBUG does not do is validate any intended relationships between the computed intervals, and its analysis scope is confined to a single contract and a single transaction.

INTENTCHECKER takes SOLQDEBUG as its foundation and adds three things. First, it introduces an *intent annotation model*—two annotation types, @During and @Post, together with a grammar of clauses that lets developers declare what numeric relationships should hold at specific program points and after function execution. Second, it introduces a *validation engine* that reads the intervals computed by the extended abstract interpreter, compares them against the declared intent, and produces a graded result: *Satisfied* when the intent holds for all values in the intervals, *Violated* when it fails for all values, and *Warning* when it holds for some and fails for others. Each result is accompanied by a risk score on a 0–10 scale and, for non-*Satisfied* results, by the sub-interval in which the intent may be violated. Third, it *extends the analysis scope* to cover multi-contract single-transaction code: inherited contracts, libraries invoked via `using` directives and `delegatecall`, interface-typed variables, and a practically useful arithmetic subset of inline assembly. To handle return values from interface calls whose implementation code is unavailable, we introduce @IReturn, a new debug annotation that uses the same “inject a range at an unknown point” semantics already used by SOLQDEBUG’s existing @StateVar and @LocalVar annotations.

Given an intent clause $l \text{ op } r$ where l ranges over interval L and r ranges over interval R , the validation result is deterministically categorized into one of three states:

Satisfied $\iff \forall l \in L, \forall r \in R : l \text{ op } r \text{ holds}$

Violated $\iff \forall l \in L, \forall r \in R : l \text{ op } r \text{ fails}$

Warning $\iff$ neither Satisfied nor Violated

Unlike warnings from pattern-based static analyzers, which are heuristic indicators that a known buggy shape may be present, INTENTCHECKER’s *Warning* is a precise statement: the declared intent holds for some combinations of values in L and R but not others, and the tool reports exactly which sub-intervals fall into the violating region. In this sense INTENTCHECKER is best understood as an *awareness tool* rather than a detector: its goal is not to pronounce “this is a bug” but to bring the developer’s attention to the specific program points where the abstract execution cannot confirm the declared intent. The next section describes the SOLQDEBUG foundation and the scope extensions in detail, and Section 4 presents the intent annotation grammar and the validation engine.

3 Preliminaries: SolQDebug Foundations and Scope Extension

This section reviews the parts of SOLQDEBUG (Jeon et al., 2025) that INTENTCHECKER relies on and describes the scope extensions we introduce to support intent validation over realistic multi-contract Solidity code. Readers familiar with SOLQDEBUG may skip §3.1 and proceed directly to §3.2.

3.1 SolQDebug Recap

SOLQDEBUG is an edit-time debugging assistant for Solidity that provides statement-level feedback through interval-domain abstract interpretation. It addresses two forms of inefficiency in traditional Solidity debugging: the temporal cost of the compile–deploy–setup–trace cycle in tools such as Remix IDE, and the iterative cost imposed by the non-reversible state model of the EVM. Both inefficiencies are resolved by performing the entire analysis inside the editor, without contacting any blockchain node.

Interactive parsing and incremental CFG. SOLQDEBUG extends the Solidity ANTLR grammar with seven specialized entry rules that accept partial code fragments—a lone `else` branch, a function header without a body, a single statement inside an existing block, and so on. As the developer types or edits code, only the affected region is re-parsed, and the corresponding nodes are inserted into an *incremental control-flow graph* via a line-to-node index. Abstract interpretation then resumes from the edit point, reusing previously computed intervals for nodes whose inputs have not changed.

Debug annotations and the “unknown value, injected range” semantics. Because abstract interpretation over completely unconstrained inputs produces overly conservative results, SOLQDEBUG lets developers supply range information for variables whose actual values are unknown to the analyzer. Two debug annotation families cover this role:

- **@StateVar**—for global and state variables whose values are determined by prior transactions (e.g., `totalSupply`, a user’s balance, a lending pool’s reserves). SOLQDEBUG has no way to know what value these variables hold at the start of the function being debugged; the developer declares a representative range instead.
- **@LocalVar**—for function parameters and locally declared variables whose values are determined by the caller or by earlier computation. This is how the developer specifies the test cases under which the function should be analyzed.

Both annotations share a single, unifying semantics: at each program point where the analyzer would otherwise have only $\top$ (“any value”), the developer explicitly injects a bounded range. The analyzer then proceeds to compute intervals for downstream program points under that injected range, giving the developer a line-by-line view of how the function behaves for the chosen inputs.

Output semantics: range in, range out. SOLQDEBUG’s output for each annotated path is a table of intervals—one per variable per line—reflecting what the abstract

execution computed. SOLQDEBUG itself makes no judgment about whether these intervals are “correct”: it neither compares them against any specification nor flags any of them as buggy. It is, in the strict sense, a debugger: it shows the developer what happens, and leaves the interpretation to the developer. INTENTCHECKER’s validation engine is precisely the layer that adds such interpretation on top of this “range in, range out” foundation.

Analysis scope. SOLQDEBUG’s published scope is deliberately narrow: *single contract*, *single transaction*. A function in the target contract is analyzed under the assumption that its state variables hold the values declared by `@StateVar`, and that only that function executes. Inter-contract calls, inheritance, and library invocations are explicitly out of scope and are left to future work in the SOLQDEBUG paper. The next subsection describes how INTENTCHECKER extends this scope to handle the realistic multi-contract code that real numeric logic errors tend to live in.

3.2 Scope Extension for Multi-Contract Single-Transaction Analysis

Real-world numeric logic errors rarely occur in single-contract code: most of the Web3Bugs and NumScout benchmark cases involve inherited contracts, library calls, or interface-typed variables. Validating intent over such code requires extending SOLQDEBUG’s analysis scope from *single contract* to *multi-contract single-transaction*, where a single atomic transaction may touch several contracts located in the same source file or its immediate dependencies, but is still confined to one execution step. This subsection describes the four extensions required to make this scope practical: (a) a deployment-unit model for identifying which contracts share storage with the target, (b) library and `using`-directive handling, (c) interface-typed variables and the `@IReturn` debug annotation, and (d) a practically useful subset of inline assembly.

(a) Deployment-unit scope. INTENTCHECKER treats as “in-scope” everything that shares storage with the target contract in a single EVM execution: the target contract itself, all inherited parents resolved via C3 linearization, and any libraries invoked through `delegatecall` via Solidity’s `using for` directive. These all run under the same `address(this)` and over the same storage, so debug annotations on the target contract’s state variables can be reused without reinterpretation whenever the analyzer steps into an inherited function or a library function. External contracts—i.e., separately deployed contracts invoked through an address or an interface-typed variable—are *out-of-scope* by default, because their storage is a different deployment unit that SOLQDEBUG’s debug annotations cannot reach. Extension (c) below describes how `@IReturn` allows a restricted form of data injection across this deployment boundary for view and pure interface calls.

(b) Library and interface handling. For each Solidity construct in the target source file, INTENTCHECKER pre-builds a dedicated CFG: contracts are stored as `ContractCFG`, libraries as `LibraryCFG`, and interfaces as `ContractCFG` instances marked with a dedicated flag whose functions consist of an empty `entry-to-exit` body (interfaces carry no implementation). When a `using LibraryName for T` directive is encountered, the corresponding `LibraryCFG` is attached to the declaring contract’s

set of using-libraries, and subsequent method-call resolution on values of type T walks this set to find the matching library function overload. Inherited functions are resolved via the same C3 linearization order used by the Solidity compiler, ensuring that overrides behave as they would at runtime. Library function bodies are analyzed in-place, reusing the debug annotations of the target contract since a library invoked via `delegatecall` shares storage with its caller.

(c) @IReturn: Interface Return Value Debug Annotation. Interface-typed variables (e.g., `IOracle oracle`) present a structural problem that debug annotations on state or local variables cannot solve: the implementation code behind the interface is not present in the source file at all, so SOLQDEBUG’s abstract interpreter must return $\top$ for any value derived from an interface call. This $\top$ then propagates into every downstream computation, defeating the purpose of intent validation for any function that reads an oracle or queries an external token contract.

Our observation is that this situation is semantically identical to the case that `@StateVar` and `@LocalVar` already handle. In all three cases, the analyzer encounters a program point at which the actual value is unknown because its origin lies outside the analysis scope: `@StateVar` handles unknown storage values left behind by prior transactions, `@LocalVar` handles unknown inputs supplied by the caller, and a fourth kind of unknown—the return value of an interface call—can be handled by the same “inject a developer-specified range at the unknown point” mechanism. This is consistent with the underlying philosophy of debugging: the developer chooses a particular execution path to inspect, declares the inputs relevant to that path, and observes the line-by-line output. An interface return value is simply another input to the path under analysis; it only happens to originate from beyond a deployment-unit boundary rather than from a local variable or a storage slot.

We formalize this intuition as `@IReturn`, a new debug annotation that lets the developer bind a range to the return value of a specific interface function. Two surface syntaxes are supported:

- **Pattern A (variable-qualified).** `// @IReturn oracle.getPrice() = [1000, 2000]` binds the return value of `getPrice` on an interface-typed variable (`oracle` must be declared as some interface type).
- **Pattern B (explicit cast).** `// @IReturn IOracle(oracleAddr).getPrice() = [1000, 2000]` binds the return value of `getPrice` when the call is written as an explicit cast of an address variable to an interface type.

Both patterns may target structured return values, including struct members (`...getReserves().reserve0`) and tuple elements (`...getPair()[0]`), which we represent as an access chain so that a single return value can be partially specified and partially left as $\top$.

View/pure restriction. `@IReturn` is restricted to *view* and *pure* interface functions. The reason is that a state-modifying interface call would, in general, alter the state of the external contract whose implementation we do not have, and INTENTCHECKER has no principled way to model the resulting storage mutation in the external deployment unit. Allowing `@IReturn` on state-modifying calls would let the developer specify the return value but silently omit any side effect, producing a model that does not

Algorithm 1 ProcessIReturn: Annotation Registration

```
1: Input: annotation contractVar.func().chain = value  
2: Output: updated IReturn registry  $\mathcal{R}$ , or error  
  
3:  $I \leftarrow$  resolve contractVar to its declared interface type  
4: if  $I$  is undefined then return ERROR(“not an interface variable”)  
5: end if  
6:  $f \leftarrow$  look up func in interface  $I$   
7: if  $f$  is undefined then return ERROR(“function not in interface”)  
8: end if  
9: if  $f$  is not view or pure then  
10:   return ERROR(“@IReturn restricted to view/pure”)  
11: end if  
12:  $\mathcal{R}[(contractVar, func, chain)] \leftarrow value$ 
```

correspond to any real execution. Restricting **@IReturn** to view/pure functions avoids this unsoundness: by definition, a view or pure call has no side effects, so binding its return value is a complete description of its observable behavior.

Registration. Algorithm 1 describes how an **@IReturn** annotation is validated and registered. Like **@StateVar**, **@LocalVar**, and **@GlobalVar**, **@IReturn** annotations are processed as part of the batch annotation phase: the developer writes all debug annotations, and the analyzer then interprets the target function from its start block with all registered values in effect. The key difference is the view/pure check (line 4): if the target function is state-modifying, registration is rejected with a diagnostic, preventing the unsound model described above.

Resolution at call sites. When the abstract interpreter encounters an interface function call during analysis, it must decide whether to return $\top$ (no information) or a developer-specified value. Algorithm 2 describes the resolution procedure. The registered entries from Algorithm 1 are looked up as a short-circuit check *before* the interpreter would otherwise default to $\top$. Two surface patterns are tried in order: Pattern A matches by the variable name that holds the interface reference (e.g., `oracle.getPrice()`), and Pattern B matches by the explicit interface cast (e.g., `IOracle(addr).getPrice()`). If a match is found, the registered value is assembled—potentially following an access chain for struct members or tuple indices—and returned immediately, preventing $\top$ from propagating into downstream computations.

(d) Arithmetic-focused inline assembly support. Solidity’s inline assembly (`assembly { ... }`) is written in Yul and frequently appears in arithmetic-heavy code, particularly in high-precision fixed-point libraries such as FullMath or ABD-KMath. SOLQDEBUG originally treated any inline assembly block as a single opaque region, leaving all affected variables at $\top$. INTENTCHECKER implements a dedicated Yul visitor that recognizes the subset of Yul used for arithmetic computation and translates it into the same expression IR used by the rest of the analyzer. Concretely, the visitor supports variable declarations (`let x := expr`) and assignments (`var :=`

Algorithm 2 ResolveIReturn: Interface Call Value Resolution

```
1: Input: interface call expression  $e = \text{base.func}(\dots)$ , IReturn registry  $\mathcal{R}$ 
2: Output: resolved value or  $\top$ 

    // Pattern A: variable-qualified (e.g.,  $\text{oracle.getPrice}()$ )
3: if  $\text{base}$  is an identifier with interface type  $I$  then
4:    $\text{entries} \leftarrow \{c \mapsto v \mid (\text{base}, \text{func}, c) \mapsto v \in \mathcal{R}\}$ 
5:   if  $\text{entries} \neq \emptyset$  then
6:     return ASSEMBLE( $I, \text{func}, \text{entries}$ )
7:   end if
8: end if

    // Pattern B: explicit cast (e.g.,  $\text{IERC20(addr).balanceOf}()$ )
9: if  $\text{base}$  is a cast  $I(\text{addr})$  then
10:   $\text{entries} \leftarrow \{c \mapsto v \mid (I, \text{addr}, \text{func}, c) \mapsto v \in \mathcal{R}\}$ 
11:  if  $\text{entries} \neq \emptyset$  then
12:    return ASSEMBLE( $I, \text{func}, \text{entries}$ )
13:  end if
14: end if

15: return  $\top$  ▷ no registry match; fall back to default

16: function ASSEMBLE( $I, \text{func}, \text{entries}$ )
17:   if  $\text{entries}$  contains only the empty chain  $()$  then
18:     return  $\text{entries}[]$  ▷ single scalar return
19:   end if
20:    $\text{retType} \leftarrow$  load return type schema of  $I.\text{func}$  from interface CFG
21:    $\text{val} \leftarrow \top_{\text{retType}}$  ▷ initialize with  $\top$  of the declared type
22:   for all  $(c, v) \in \text{entries}$  do
23:     set  $\text{val}$  at access chain  $c$  to  $v$  ▷ struct member or tuple index
24:   end for
25:   return  $\text{val}$ 
26: end function
```

`expr`) inside a `yulBlock`, and maps the following Yul built-ins to their Solidity operator counterparts: arithmetic `add`, `sub`, `mul`, `div`/`sdiv`, `mod`/`smod`, `exp`; comparison `lt`/`gt`/`slt`/`sgt`/`eq`; bitwise `and`, `or`, `xor`; shifts `shl`, `shr`, `sar`; the unary `not` and `iszero`; and the three-argument `mulmod` and `addmod`, which are expanded into the corresponding Solidity expressions $(a \cdot b) \bmod n$ and $(a + b) \bmod n$. This coverage is deliberately chosen to capture the arithmetic that high-precision libraries typically express in Yul to bypass Solidity’s overflow checks, while avoiding the control-flow and memory features (`if`, `for`, `switch`, `mload`/`mstore`, `sload`/`sstore`) whose abstract semantics would require separate engineering beyond the scope of this paper. Yul built-ins outside the supported subset (e.g., `keccak256`, `call`, `log0`) are handled conservatively by leaving their results at $\top$, preserving soundness. Section 6 discusses why

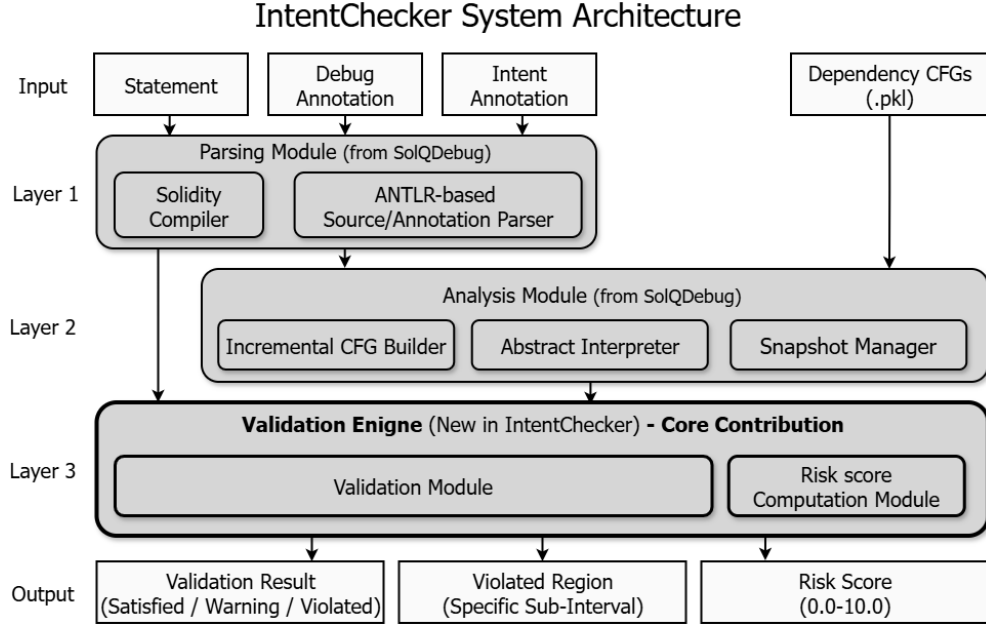


Fig. 1: System architecture of IntentChecker. Layers 1–2 (Parsing and Analysis Modules) are reused from SOLQDEBUG; Layer 3 (Validation Module) is the core contribution of this work. The Validation Engine receives parsed intent clauses from the Parsing Module and computed intervals from the Analysis Module.

we chose this particular subset and what the remaining unsupported features mean for the analysis.

Summary. Taken together, these four extensions transform SOLQDEBUG’s single-contract single-transaction sandbox into an analysis scope that covers the ways modern Solidity contracts are actually structured: multiple contracts per source file, library-mediated arithmetic, oracle-style interface calls, and hand-written Yul for performance-critical fixed-point math. All extensions preserve SOLQDEBUG’s “range in, range out” character; none of them introduce a notion of correctness on their own. Building the intent validation layer on top of this extended analyzer is the subject of the next section.

4 The Design of IntentChecker

4.1 System Architecture

Figure 1 illustrates the overall architecture of IntentChecker.

Input. IntentChecker receives three types of input:

- **Statement**—individual Solidity statements provided incrementally as the developer writes code.

- **Intent Annotation**—`@During` and `@Post` specifications expressing the developer’s intended numerical relationships.
- **Debug Annotation**—variable initialization directives that provide concrete or symbolic value ranges at program points where the actual value is unknown to the analyzer. This category includes SOLQDEBUG’s original `@GlobalVar` (constructor-initialized storage variables), `@StateVar` (storage variables modified at runtime), and `@LocalVar` (function parameters and local variables), as well as the `@IReturn` annotation introduced in §3.2(c) for interface call return values.

Output. For each intent annotation, IntentChecker produces:

- **Result**—a validation state of `Satisfied`, `Violated`, or `Warning`.
- **Risk Score**—a 0–10 scale score: 0.0 for `Satisfied`, 10.0 for `Violated`, and 0.1–9.9 for `Warning` reflecting the severity of the potential violation.
- **Violated Region**—the specific sub-intervals where the intent may be violated, helping developers identify problematic input ranges.

Internal Modules. The system builds upon SOLQDEBUG (Jeon et al., 2025), whose Parsing Module, Analysis Module, and Snapshot Manager are reused as the basis for INTENTCHECKER’s extended analysis. The Parsing Module uses ANTLR for syntax analysis of both Solidity source fragments and intent/debug annotation comments, and delegates to the official Solidity compiler for semantic validation. The Analysis Module performs incremental abstract interpretation over the interval domain, updating the control-flow graph and variable states as each statement is processed, and handles the multi-contract scope extensions (inheritance, library resolution via `using`, interface-typed variables, and the arithmetic Yul subset) described in §3.2. For every library and interface referenced from the target source file, the Analysis Module either loads a pre-analyzed *Dependency CFG*—a serialized `LibraryCFG` or `ContractCFG` object produced once and cached—or constructs one on-the-fly when the source is locally available. When a debug annotation is received, the Snapshot Manager preserves the current analysis state, allowing the system to apply temporary variable bindings, re-analyze affected code paths, and restore the original state afterwards; this mechanism is reused without modification for the new `@IReturn` annotation, since injecting a range at an interface call return site is mechanically the same as injecting one at a `@StateVar` declaration.

The Validation Engine (Layer 3) is INTENTCHECKER’s core contribution on top of this extended analyzer. It receives two inputs from the lower layers: *parsed intent clauses* from the Parsing Module, which encode the relational constraints declared in `@During/@Post` annotations, and *computed variable intervals* from the Analysis Module, which represent the abstract state of each variable at the annotated program point. For each clause, it evaluates whether the declared intent holds by comparing the actual intervals against the specified constraints, producing the validation result, risk score, and violated regions as described above.

4.2 Motivating Examples

This section illustrates how INTENTCHECKER addresses representative numeric logic errors, using three real-world cases from our evaluation benchmark (§5). The first two cases demonstrate mid-execution checks via `@During` annotations; the third demonstrates a post-execution check via `@Post`.

4.2.1 Operator Order Issue (BoostToken)

Listing 1 shows a fee-distribution function that divides before multiplying, causing integer truncation to silently reduce the transferred amount. The developer intends the marketing wallet to receive $\lfloor \text{amount} \times 5/12 \rfloor$, but `amount.div(12).mul(5)` truncates `amount/12` first, losing precision when `amount` is not a multiple of 12.

Listing 1: BoostToken: operator order issue in fee distribution (simplified).

```
1 function sendETHToTeam(uint256 amount) private {
2     // @LocalVar amount = [68, 68]
3     _devWalletAddress.transfer(amount.div(4));
4     // @During transfer.arg[0] >= amount * 5 / 12
5     _marketingWalletAddress.transfer(amount.div(12).mul(5));
6     // @During transfer.arg[0] >= amount * 2 / 9
7     _dipWalletAddress.transfer(amount.div(9).mul(2));
8     _marketingWalletAddress2.transfer(amount.div(9));
9 }
```

The `@During transfer.arg[0] >= amount * 5 / 12` annotation expresses the developer's intent that the actual transfer argument should be at least the mathematically correct share. With `amount=68`, the correct value is $\lfloor 68 \times 5/12 \rfloor = 28$, but `68.div(12).mul(5)` computes $\lfloor 68/12 \rfloor \times 5 = 5 \times 5 = 25$. INTENTCHECKER evaluates the annotation to $[25, 25] \geq [28, 28]$ and reports *Violated* with risk score 10.0.

4.2.2 Exchange Rate Validation (Nokon)

Listing 2 shows a token purchase function where operator precedence causes the buyer to receive fewer tokens than the exchange rate warrants.

Listing 2: Nokon: exchange rate calculation with operator order issue (simplified).

```
1 function buy() payable {
2     // @GlobalVar msg.value = [50000000000500000, ...]
3     // @StateVar presell = true
4     require(presell, "presell is closed");
5     uint256 amountToBuy = msg.value / ethRateFix
6                         * calculateRate();
7     // @During amountToBuy * ethRateFix >= msg.value * 250000
8     balances[msg.sender] += amountToBuy;
9 }
```

The developer's intent is that `amountToBuy` should satisfy the exchange-rate invariant: the tokens received, times the rate denominator, should be at least the ETH paid times the rate numerator. Because `msg.value / ethRateFix` truncates before multiplication, the invariant is violated for non-round input amounts. The `@During` annotation captures this invariant directly, and INTENTCHECKER reports *Violated*.

4.2.3 Minting Invariant with Interface Call (Web3Bugs #78)

Listing 3 shows a rebasing vault’s `mint` function. The bug is that minted proxy tokens are computed from the total vault balance (including the just-deposited amount), which inflates the shares beyond the deposited amount when the redeem rate deviates from 1.

Listing 3: Web3Bugs #78-H-02: vault mint with inflated proxy amount (simplified).

```
1 function mint(address to, uint256 amount)
2   public override returns (uint256) {
3     // @LocalVar amount = [500e18, 500e18]
4     // @StateVar _totalSupply = [1000e18, 1000e18]
5     // @StateVar _balances[to] = [0, 0]
6     // @IReturn IERC20(baseToken).balanceOf()
7     //   = [1500e18, 1500e18]
8     uint256 _redeemRate = redeemRate();
9     require(IERC20(baseToken).transferFrom(
10      msg.sender, address(this), amount));
11     uint256 baseBalance =
12       IERC20(baseToken).balanceOf(address(this));
13     uint256 proxy = (baseBalance * ONE) / _redeemRate;
14     // @Post _balances[to] <= amount
15     _mint(to, proxy);
16 }
```

The `@Post _balances[to] <= amount` annotation expresses the invariant that no user should receive more shares than the tokens they deposited. The `@IReturn` debug annotation injects a concrete balance (1500e18) for the external `balanceOf` call, which the analyzer would otherwise evaluate as $\top$ (§3.2(c)). With these annotations, `INTENTCHECKER` computes the proxy amount and finds that it exceeds `amount`, reporting *Violated*. This example demonstrates how `@IReturn` enables intent validation for functions that depend on external contract state.

These three examples—drawn from real bugs in the evaluation benchmark—illustrate the two temporal scopes of intent validation: mid-execution checks (`@During` with relational and argument constraints) and post-execution checks (`@Post` combined with `@IReturn` for external state injection). The complete intent model supporting these annotations is defined in the following section.

4.3 Intent Model

This section defines the syntax and semantics of intent annotations. The two annotation types correspond to the two temporal scopes at which numeric intent can be validated: `@During` checks conditions at specific program points within a function’s execution, while `@Post` checks conditions after the function completes. This section specifies the concrete grammar and semantics of each construct.

Annotation Syntax Overview. Figure 2 presents the grammar of intent annotations. Intent annotations are embedded in single-line comments, allowing them to coexist with Solidity code without affecting compilation. Each annotation begins with a marker (`@During` or `@Post`) followed by one or more clauses. Multiple clauses can be combined using logical operators: `&&` (conjunction) requires all clauses to hold, while `|`— (disjunction) requires at least one clause to hold.


```

duringIntent  → // @During duringClause (logicOp duringClause)*
postIntent   → // @Post postClause (logicOp postClause)*

duringClause → intentValue (before relOp after)
                | intentValue (assign relOp current)
                | identifier .arg[ n ] relOp intentValue
                | commonClause

postClause   → intentValue (entry relOp exit)
                | commonClause

commonClause → returnExpression relOp intentValue
                | return[n] relOp intentValue
                | intentValue relOp PercentOf( intentValue , p )
                | intentValue relOp ceil( intentValue , d )
                | intentValue relOp floor( intentValue , d )
                | intentValue relOp intentValue
                | intentValue => intentValue

intentValue  → arithExpr
arithExpr   → arithExpr (+ | -) arithTerm | arithTerm
arithTerm   → arithTerm (* | / | %) arithFactor | arithFactor
arithFactor → number | [ number , number ] | varRef | ( arithExpr )
varRef      → identifier subAccess*
subAccess   → . identifier | [ expr ]

logicOp     → && | ||
relOp       → < | > | <= | >= | == | != | in | not in

```

Fig. 2: Grammar of intent annotations.

Intent values within clauses support arithmetic expressions using standard operators (+, -, *, /, %), variable references with member and index access (e.g., `balances[msg.sender]`), numeric literals, and inline intervals written as `[a, b]` to represent value ranges directly. Relational operators include `<`, `>`, `<=`, `>=`, `==`, `!=`, as well as `in` and `not in` for interval membership tests.

@During Annotation. The `@During` annotation specifies intended relationships at specific program points within a function’s execution. It supports the following clause types:

- `x(before relOp after)`—compares the value of variable `x` immediately before and after an assignment statement. This validates that assignments modify variables as intended (e.g., `balance(before > after)` asserts that the balance decreases).
- `x(assign relOp current)`—compares the value being assigned to `x` against its current value. This is useful for validating incremental updates (e.g., `counter(assign == current + 1)`).
- `func.arg[n] relOp value`—validates the n -th argument passed to a function call, ensuring that function calls receive arguments satisfying specified constraints before execution proceeds.

@Post Annotation. The `@Post` annotation specifies intended relationships that should hold after function execution completes. It supports the following clause types:

- `x(entry relOp exit)`—compares the value of variable `x` at function entry against its value at function exit. This validates function-level effects on state variables (e.g., `totalSupply(entry == exit)` asserts that total supply remains unchanged).

Common Clauses. Both `@During` and `@Post` annotations share the following clause types:

- `returnExpression relOp value`—validates the expression being returned against a specified value or range.
- `return[n] relOp value`—for functions returning multiple values (tuples), validates the n -th return value.
- `x relOp PercentOf(y, p)`—validates that `x` equals `p` percent of `y`, accounting for integer arithmetic. This is particularly useful for fee calculations and token distributions.
- `x relOp ceil(y, d)` and `x relOp floor(y, d)`—validate rounding behavior, where `d` specifies the divisor. These clauses help developers express intended rounding directions in division operations.
- `x relOp y`—general relational comparison between two intent values.
- `x => y`—implication clause, asserting that if condition `x` holds, then condition `y` must also hold. This enables conditional intent specifications.

4.4 Validation Engine

The Validation Engine validates developer-specified intent annotations against actual program behavior computed through abstract interpretation. Given the CFG-based interval analysis from Section 4, the engine evaluates each annotation clause and determines whether the intended relationship holds across all possible execution states.

To perform validation, the engine captures variable environments at specific program points:

- *before_env*: variable state immediately before a statement executes
- *after_env*: variable state immediately after a statement executes (referred to as *current* in assign clauses)
- *assign_env*: the value being assigned in an assignment statement
- *entry_env*: variable state at function entry
- *exit_env*: variable state at function exit, computed by joining all normal termination paths

4.4.1 Validation Targets

@During Validation. The `@During` annotation is validated immediately after the associated statement executes:

- `var(before op after)`: compares *before_env* against *after_env* for variable `var`.
- `var(assign op current)`: compares *assign_env* against the current value of `var`.
- `func.arg[n] op value`: validates the n -th argument passed to the function call.

@Post Validation. The @Post annotation is validated when function execution completes:

- `var(entry op exit)`: compares `entry_env` against `exit_env` for variable `var`.

Common Clauses. The following clauses can be used in both @During and @Post annotations:

- `returnExpression op value`: compares the return value against the evaluated value.
- `return[n] op value`: compares the n -th element of a tuple return value against value.
- `exp1 op PercentOf(exp2, p)`: compares `exp1` against $p\%$ of `exp2`.
- `exp1 op ceil(exp2, d)` and `exp1 op floor(exp2, d)`: compares `exp1` against `exp2` rounded up or down to the nearest multiple of `d`.
- `exp1 op exp2`: general comparison between two expressions.

4.4.2 State Determination and Risk Scoring

The engine compares interval-domain values and produces one of three possible outcomes:

- *satisfied*: the condition holds for all values in the intervals.
- *violated*: the condition fails for all values in the intervals.
- *warning*: the condition holds for some values but not others.

Each result is mapped to a *risk score* on a 0–10 scale. A score of 0.0 indicates a fully satisfied condition, while 10.0 indicates a definite violation. For *warning* results, the score reflects the severity of the potential violation. The engine classifies warnings into three risk types based on the structure of the violated region: type 1 (single-direction, minor) maps to scores 0.1–3.3; type 2 (single-direction violation) maps to 3.4–6.6; and type 3 (bidirectional violation or equality operators) maps to 6.7–9.9. Within each band, the score is computed as $base + (1 - p_{true}) \times span$, where p_{true} is the probability that the condition holds under a uniform distribution assumption.

Clause-to-Interval Mapping. Table 2 shows how each clause type maps to the interval comparison inputs L and R in Algorithm 3.

Interval Comparison Algorithm. Algorithm 3 presents the interval comparison procedure. Given two integer intervals $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$ with an operator op , the algorithm partitions L into three regions under a uniform distribution assumption over both intervals:

- T_s (*satisfied region*): values of L for which $l op r$ holds for all $r \in R$.
- T_v (*violated region*): values of L for which $l op r$ fails for all $r \in R$.
- T_w (*warning region*): values of L whose truth value depends on the actual value of R .

Table 3 defines the early-return conditions and region sizes for each operator, where $n_L = l_{\max} - l_{\min} + 1$ and $n_R = r_{\max} - r_{\min} + 1$ denote the number of integer values in each interval.

Table 2: Mapping of clause types to interval comparison inputs. Here, exp_1 and exp_2 denote evaluated intent expressions; *return value* is the value returned by the current return statement; *return value*[n] is the n -th element when the function returns a tuple.

Clause Type	L	R
<code>var(before op after)</code>	$before_env[\mathbf{var}]$	$after_env[\mathbf{var}]$
<code>var(assign op current)</code>	$assign_env[\mathbf{var}]$	$after_env[\mathbf{var}]$
<code>func.arg[n] op value</code>	$\mathbf{arg}[n]$	$\mathbf{value}$
<code>var(entry op exit)</code>	$entry_env[\mathbf{var}]$	$exit_env[\mathbf{var}]$
<code>returnExpression op value</code>	$return\ value$	$\mathbf{value}$
<code>return[n] op value</code>	$return\ value[n]$	$\mathbf{value}$
$exp_1\ op\ exp_2$	exp_1	exp_2

Table 3: Per-operator satisfied/violated conditions and region sizes. All size expressions are clamped to $\max(0, \cdot)$. For `in/not in`, $|L \cap R|$ denotes the count of L -values within the range of R .

op	Satisfied condition	Violated condition	$ T_s $	$ T_v $
$>$	$l_{\min} > r_{\max}$	$l_{\max} \leq r_{\min}$	$l_{\max} - r_{\max}$	$r_{\min} - l_{\min} + 1$
$\geq$	$l_{\min} \geq r_{\max}$	$l_{\max} < r_{\min}$	$l_{\max} - r_{\max} + 1$	$r_{\min} - l_{\min}$
$<$	$l_{\max} < r_{\min}$	$l_{\min} \geq r_{\max}$	$r_{\min} - l_{\min}$	$l_{\max} - r_{\max} + 1$
$\leq$	$l_{\max} \leq r_{\min}$	$l_{\min} > r_{\max}$	$r_{\min} - l_{\min} + 1$	$l_{\max} - r_{\max}$
$=$	disjoint $\wedge$ singletons	disjoint	$ L \cap R $	$n_L - L \cap R $
$\neq$	disjoint	disjoint $\wedge$ singletons	$n_L - L \cap R $	$ L \cap R $
<code>in</code>	$L \subseteq R$	$L \cap R = \emptyset$	$ L \cap R $	$n_L - L \cap R $
<code>not in</code>	$L \cap R = \emptyset$	$L \subseteq R$	$n_L - L \cap R $	$ L \cap R $

For the comparison operators ($>$, $\geq$, $<$, $\leq$), the warning region T_w contains values of L whose comparison result depends on the actual value of R . Since Solidity operates on integers, for each $l \in T_w$ the number of R -values satisfying $l\ op\ r$ forms an arithmetic sequence across the warning region. The expected true count E_w is therefore computed in closed form as $E_w = (a + b) \cdot |T_w| / (2 \cdot n_R)$, where a and b are the number of satisfying R -values for the smallest and largest elements of T_w , respectively.

For equality and inequality operators ($=$, $\neq$), each overlapping L -value has a $1/n_R$ probability of matching any particular R -value, rather than being a definite match. The probability is therefore adjusted by the factor $1/n_R$.

Compound Clauses and Implication. Multiple clauses can be combined using logical operators. The engine evaluates each clause independently and combines the three-valued results according to Table 4.

Algorithm 3 Interval Comparison with Risk Score

```
1: Input: Intervals  $L = [l_{\min}, l_{\max}]$ ,  $R = [r_{\min}, r_{\max}]$ , operator  $op$ 
2: Output: ( $state$ ,  $risk\_score$ )
3:  $n_L \leftarrow l_{\max} - l_{\min} + 1$ ,  $n_R \leftarrow r_{\max} - r_{\min} + 1$ 

    // Step 1: Early return for definite cases (Table 3)
4: if SATISFIEDCOND( $op$ ,  $L$ ,  $R$ ) then
5:     return ( $satisfied$ , 0.0)
6: else if VIOLATEDCOND( $op$ ,  $L$ ,  $R$ ) then
7:     return ( $violated$ , 10.0)
8: end if

    // Step 2: Region partitioning (Table 3)
9:  $|T_s|$ ,  $|T_v| \leftarrow$  per-operator formulas from Table 3
10:  $|T_w| \leftarrow n_L - |T_s| - |T_v|$ 

    // Step 3: Compute  $p_{true}$ 
11: if  $op \in \{\text{in}, \text{not in}\}$  then
12:      $p_{true} \leftarrow |T_s| / n_L$ 
13: else if  $op \in \{=, \neq\}$  and  $n_R > 1$  then
14:      $p_{true} \leftarrow |T_s| / (n_L \cdot n_R)$   $\triangleright = \text{case}; \neq: 1 - |T_v| / (n_L \cdot n_R)$ 
15: else  $\triangleright >, \geq, <, \leq$ 
16:      $E_w \leftarrow$  expected true count from  $T_w$  via arithmetic series over  $R$ 
17:      $p_{true} \leftarrow (|T_s| + E_w) / n_L$ 
18: end if

    // Step 4: Risk score
19:  $risk\_type \leftarrow$  CLASSIFYRISK( $false\_regions$ ,  $op$ )
20:  $risk\_score \leftarrow base(risk\_type) + (1 - p_{true}) \times span(risk\_type)$ 
21: return ( $warning$ ,  $risk\_score$ )
```

4.4.3 Algorithm Verification

We establish the correctness of Algorithm 3 in two complementary ways: a mathematical soundness proof showing that the closed-form expressions in Table 3 compute p_{true} exactly, and an empirical cross-validation against brute-force ground truth across a comprehensive battery of test cases.

Definitions. Let $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$ be nonempty integer intervals with $n_L = l_{\max} - l_{\min} + 1$ and $n_R = r_{\max} - r_{\min} + 1$. For an operator op , let

$$N_{sat}(L, R, op) = |\{(l, r) \in L \times R : l \text{ op } r\}|$$

Table 4: Three-valued logic combination rules for compound clauses. S = *satisfied*, V = *violated*, W = *warning*.

$\&\&$	S	V	W
S	S	V	W
V	V	V	V
W	W	V	W
$\parallel$	S	V	W
S	S	S	S
V	S	V	W
W	S	W	W
$\Rightarrow$	S	V	W
S	S	V	W
V	S	S	S
W	W	W	W

denote the number of satisfying pairs. Under a uniform distribution over $L \times R$, the true probability of satisfaction is

$$p_{true}(L, R, op) = \frac{N_{sat}(L, R, op)}{n_L \cdot n_R}.$$

Algorithm 3 is *sound* if its returned value p_{func} equals p_{true} for every well-formed input.

Region partition. For $op \in \{>, \geq, <, \leq\}$, the algorithm decomposes L into three disjoint regions instantiating the prose given earlier:

$$\begin{aligned} T_s &= \{l \in L : l \text{ op } r \text{ holds for all } r \in R\}, \\ T_v &= \{l \in L : l \text{ op } r \text{ fails for all } r \in R\}, \\ T_w &= L \setminus (T_s \cup T_v). \end{aligned}$$

The decomposition is exhaustive: for fixed $l \in L$, the truth value of $l \text{ op } r$ is monotonic in r , so it is either always true ($l \in T_s$), always false ($l \in T_v$), or transitions exactly once as r ranges over R ($l \in T_w$).

Lemma 1 (Region sizes for $>$) *For $op = >$, in the warning case ($l_{\min} \leq r_{\max}$ and $l_{\max} > r_{\min}$),*

$$|T_s| = \max(0, l_{\max} - r_{\max}), \quad |T_v| = \max(0, r_{\min} - l_{\min} + 1).$$

Proof $T_s = \{l \in L : l > r_{\max}\} = L \cap [r_{\max} + 1, \infty) = [\max(l_{\min}, r_{\max} + 1), l_{\max}]$, whose cardinality is $\max(0, l_{\max} - r_{\max})$. Similarly, $T_v = L \cap (-\infty, r_{\min}] = [l_{\min}, \min(l_{\max}, r_{\min})]$, with cardinality $\max(0, r_{\min} - l_{\min} + 1)$. $\square$

Lemma 2 (Sum over T_w for $>$) *Under the same assumptions, let $T_w = [l_w^-, l_w^+]$ and define $a = l_w^- - r_{\min}$, $b = l_w^+ - r_{\min}$. Then*

$$\sum_{l \in T_w} |\{r \in R : l > r\}| = \frac{(a+b)|T_w|}{2}.$$

Proof For $l \in T_w$ we have $r_{\min} < l \leq r_{\max}$, so $\{r \in R : r < l\} = [r_{\min}, l-1]$, with cardinality $l - r_{\min}$. Summing over $l \in T_w$ yields an arithmetic progression:

$$\sum_{l=l_w^-}^{l_w^+} (l - r_{\min}) = \sum_{k=a}^b k = \frac{(a+b)(b-a+1)}{2} = \frac{(a+b)|T_w|}{2}. \quad \square$$

Theorem 3 (Soundness for $>$) *For $op = >$, $p_{func} = p_{true}$.*

Proof Case 1 ($l_{\min} > r_{\max}$). Every pair satisfies $l > r$, so $N_{sat} = n_L \cdot n_R$ and $p_{true} = 1$. Algorithm 3 takes the satisfied early return and outputs $p_{func} = 1$.

Case 2 ($l_{\max} \leq r_{\min}$). No pair satisfies $l > r$, so $N_{sat} = 0$ and $p_{true} = 0$. The algorithm takes the violated early return and outputs $p_{func} = 0$.

Case 3 (warning case). Decompose

$$N_{sat} = \sum_{l \in L} |\{r \in R : l > r\}| = \underbrace{|T_s| \cdot n_R}_{l \in T_s} + \underbrace{\sum_{l \in T_w} (l - r_{\min})}_{l \in T_w} + \underbrace{0}_{l \in T_v},$$

where the first term uses the fact that every $l \in T_s$ satisfies $l > r$ for every $r \in R$. By Lemma 2, the middle sum equals $(a+b)|T_w|/2$. Dividing by $n_L \cdot n_R$ gives

$$p_{true} = \frac{|T_s|}{n_L} + \frac{(a+b)|T_w|}{2n_L n_R} = \frac{|T_s| + E_w}{n_L},$$

where $E_w = (a+b)|T_w|/(2n_R)$ is exactly the quantity computed by Algorithm 3. Hence $p_{func} = p_{true}$. $\square$

Corollary 4 (Soundness for $\geq, <, \leq$) *The remaining comparison operators $op \in \{\geq, <, \leq\}$ admit symmetric proofs: the strict inequality is replaced by its non-strict counterpart, or the roles of L and R are swapped. The closed forms of $|T_s|$ and $|T_v|$ in Table 3 arise from Lemma 1 via these substitutions, and the contribution from T_w is computed by the same arithmetic-series argument as Lemma 2, yielding $p_{func} = p_{true}$ in each case.*

Theorem 5 (Soundness for $=$ and $\neq$) *For $op \in \{=, \neq\}$, $p_{func} = p_{true}$.*

Proof For $op = =$, each value $v \in L \cap R$ contributes exactly one satisfying pair (v, v) , and no other pair satisfies $l = r$. Hence $N_{sat} = |L \cap R|$ and

$$p_{true}(=) = \frac{|L \cap R|}{n_L \cdot n_R}.$$

When $n_R > 1$, Algorithm 3 branches on $|T_s|/(n_L \cdot n_R)$, with $|T_s| = |L \cap R|$ from Table 3, matching p_{true} . The case $op \neq$ follows by complementation: $p_{true}(\neq) = 1 - p_{true}(=) = 1 - |T_s|/(n_L \cdot n_R)$, which is exactly the complementary expression noted in Table 3.

When $n_R = 1$, the algorithm bypasses the dedicated $n_R > 1$ branch and falls through to the general comparison path. The warning region T_w is then necessarily empty (since $|T_s| + |T_v| = |L \cap R| + (n_L - |L \cap R|) = n_L$), so $E_w = 0$ and the formula reduces to $|T_s|/n_L = |L \cap R|/n_L$. Direct calculation confirms this equals p_{true} when $n_R = 1$: each $l \in L$ contributes one satisfying pair iff $l = r_{min}$, giving $N_{sat} = |L \cap R|$ and $p_{true} = |L \cap R|/n_L$. $\square$

Theorem 6 (Soundness for **in** and **not in**) For $op \in \{\mathbf{in}, \mathbf{not\ in}\}$, $p_{func} = p_{true}$.

Proof The semantics of **in** treats R as a set: $l \in R$ holds iff $r_{min} \leq l \leq r_{max}$, independent of any particular r . Each $l \in L \cap R$ therefore yields n_R satisfying pairs (one for every r), so $N_{sat} = |L \cap R| \cdot n_R$ and

$$p_{true}(\mathbf{in}) = \frac{|L \cap R| \cdot n_R}{n_L \cdot n_R} = \frac{|L \cap R|}{n_L} = \frac{|T_s|}{n_L},$$

which matches the algorithm's branch for **in**. The case **not in** is the complement, $1 - p_{true}(\mathbf{in})$. $\square$

Compositional soundness. Compound clauses are built from atomic clauses via the connectives $\wedge$, $\vee$, and $\Rightarrow$, evaluated under the three-valued logic of Table 4. Treating *warning* as the third truth value “unknown,” these tables coincide with Kleene’s strong three-valued logic, a standard result whose soundness with respect to its underlying classical interpretation is well known: a compound clause evaluates to *satisfied* (resp. *violated*) only when its classical reading would be true (resp. false) under every disambiguation of *warning*. By induction on clause structure, if every atomic sub-clause is evaluated soundly by Algorithm 3, the compound clause is evaluated soundly as well.

Edge cases. Two boundary conditions deserve mention. (i) *Empty intervals.* An empty L or R corresponds to an infeasible execution path; the abstract interpreter prunes such branches before invoking Algorithm 3, so the algorithm is never called on empty inputs. (ii) *Singleton right interval.* When $n_R = 1$, the comparison operators $\{>, \geq, <, \leq\}$ reduce to standard pointwise tests handled by the early returns; the warning region T_w degenerates to at most one element, and Lemma 2 continues to hold (the arithmetic series is trivial). The equality operators $\{=, \neq\}$ with $n_R = 1$ are handled by Theorem 5. Together, Theorems 3, 5, and 6, Corollary 4, and the compositional argument above establish that Algorithm 3 computes p_{true} exactly for every operator and every well-formed input.

Empirical cross-validation. To guard against transcription errors and unforeseen edge cases in the formal development above, we additionally verified Algorithm 3

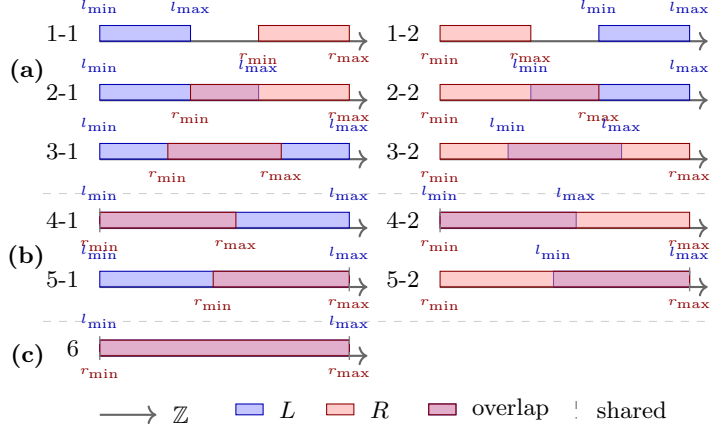


Fig. 3: Eleven topological configurations of two integer intervals, grouped by the number of shared endpoints: (a) no shared endpoints (six configurations), (b) one shared endpoint (four configurations), and (c) both shared endpoints (the identical case). Overlapping regions appear as blended color; dashed vertical lines indicate shared endpoints.

on a comprehensive battery of test cases that exhaustively cover the topological configurations of two integer intervals.

Interval topology. Given two integer intervals $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$, their relative position falls into exactly one of eleven *topological configurations*, determined by how the endpoints relate. We organize them into three groups based on the number of shared endpoints (Figure 3): (a) no shared endpoints (six configurations covering separation, partial overlap in either direction, and strict containment in either direction), (b) one shared endpoint (four configurations covering left- or right-shared with L or R being the wider interval), and (c) both shared endpoints (the identical case).

Verification method. For each of the eleven topological configurations, we generate five *width profiles* that systematically vary the interval sizes: Small (width 1–3), Medium (5–10), Large (20–50), Asymmetric-L ($|L| \gg |R|$), and Asymmetric-R ($|R| \gg |L|$). Each configuration–profile pair is tested against all eight comparison operators ($>$, $\geq$, $<$, $\leq$, $=$, $\neq$, **in**, **not in**), yielding $11 \times 5 \times 8 = 440$ test cases. For every test case, we compute the *ground-truth* probability p_{bf} by exhaustively enumerating all (l, r) integer pairs under a uniform distribution and compare it against the value p_{func} returned by Algorithm 3. A test passes when $|p_{bf} - p_{func}| < 0.001$.

Result. All 440 test cases pass with an exact match between the brute-force ground truth and the algorithm output (Table 5). The 100% pass rate across all configurations and width profiles cross-validates the soundness proof above, confirming that the closed-form expressions of Table 3 indeed compute p_{true} for every possible relative position of two integer intervals, independent of their absolute sizes.

Table 5: Brute-force verification results, grouped by topological category. Each category aggregates its constituent configurations across 5 width profiles $\times$ 8 operators per configuration.

Topology category	Tests	Pass	Fail
(a) No shared endpoints (6 configurations)	240	240	0
(b) One shared endpoint (4 configurations)	160	160	0
(c) Both shared endpoints (1 configuration)	40	40	0
Total	440	440	0

5 Evaluation

The validation algorithm itself was already verified in §4.4.3, where we proved that the closed-form expressions of Table 3 compute p_{true} exactly and cross-validated this proof against brute-force ground truth across every topological configuration of two integer intervals. The remaining questions concern INTENTCHECKER’s behavior on real-world code, the structural ceiling of its annotation model, and its position relative to existing detection tools. We address these questions through the following three RQs:

- **RQ1 (Mitigation):** When intent and debug annotations are provided based on bug report descriptions, can INTENTCHECKER surface real-world numeric logic errors from the Web3Bugs and NumScout benchmarks via its graded *Violated/Warning* signal? For instances that the tool cannot handle, what is the structural reason, and how does this distribute across the limitation types defined in our taxonomy?
- **RQ2 (Annotation Model Expressibility):** Among the instances that fail due to annotation-model limitations rather than analysis-engine limitations, what fraction can the annotation model express—and for those it can, how directly does the annotation capture the correct behavior? This question separates the model’s *expressibility ceiling* from RQ1’s empirical mitigation result, classifying each case as inexpressible, strongly expressible (the annotation encodes the correct value directly), or weakly expressible (the annotation detects that something is wrong, without stating the correct value).
- **RQ3 (Comparison with Existing Detection Tools):** How does INTENTCHECKER’s feedback compare to that of ScType, GPTScan, and NumScout on the same set of bugs, in terms of (i) the detection model each tool employs (specification-driven vs. pattern-driven), (ii) per-case analysis latency, and (iii) coverage complementarity? The goal is to characterize where each tool sits in the detection landscape, not to argue numerical superiority.

5.1 Experimental Setup

We construct a benchmark of 89 real-world numeric logic error instances drawn from three complementary sources, each contributing different error patterns under INTENTCHECKER’s scope (multi-contract / single-transaction).

NumScout (7 instances). NumScout (Chen et al., 2025) provides a curated collection of numeric logic errors mined from deployed BNB Smart Chain contracts. We use the *95-sample* subset distributed in NumScout’s official repository (Experiment/95_Samples.Run), which the authors identify as their main case-level evaluation set. We select one representative instance per pattern from the six numerical defect patterns enumerated in NumScout (Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, Minor Amount Retention, and Profit Opportunity), with two instances for Precision Loss Trend, yielding seven cases. We target contracts with compiler pragma ≥ 0.6 —the minimum version supported by INTENTCHECKER’s Solidity front-end. One selected contract carries an older 0.4.x pragma but uses only language constructs that are semantically identical under ≥ 0.6 (no checked-arithmetic difference, no breaking syntax change in the analyzed function); we treat it as compatible with the supported subset.

Flyinointment (1 instance). We additionally include one Greedy Contract instance from the dataset of Soud et al. (2024), which originally introduced the Greedy Contract pattern and is referenced in §2.1. NumScout’s pattern catalog does not include Greedy Contract, so this single instance is sourced separately to ensure pattern-level coverage of all the single-transaction error patterns described in §2.1.

Web3Bugs (81 instances). Web3Bugs (Zhang et al., 2023) systematically collected and classified 493 high-severity vulnerabilities from Code4rena audit contests. We restrict the candidate set to bugs whose execution context fits INTENTCHECKER’s scope, namely *multi-contract single-transaction* numeric logic errors. This corresponds to (i) all bugs in categories *S6-1* through *S6-4* (Erroneous Accounting, 86 bugs), whose root causes are semantic errors in arithmetic or accounting logic and which the Web3Bugs authors confirm to be single-transaction, and (ii) the subset of *S3-1* (Inconsistent State Updates) bugs whose root cause involves a numeric computation with a missing or stale state variable update. We manually reviewed the 22 *S3-1* bugs and retained the 11 instances meeting this criterion, excluding non-numeric cases such as approval resets and access-control omissions. This yields 97 candidate instances.

We then exclude 16 instances whose target contracts contain inline assembly blocks—either directly in the target contract or in locally imported/inherited files—which cannot be processed by the abstract interpreter (the supported Yul subset is described in §3.2). After these exclusions, the final Web3Bugs subset contains 81 instances spanning Erroneous Accounting (72) and Inconsistent State Updates (9).

Dataset Summary. Table 6 summarises the resulting benchmark. For each instance, we record the target contract, the buggy function, the specific lines identified in the audit report, and the error pattern.

Exclusions. Of the 89 collected instances, 14 are excluded after manual inspection because they fall outside the definition of a development-time numeric logic error. The exclusion categories are: source already silently patched (E1, 2 cases); checked-arithmetic revert under Solidity ≥ 0.8 such that the program traps rather than returns a wrong value (E2, 2 cases); duplicate audit findings on the same buggy site (E3, 2 cases); reports the original auditors disputed as intended behavior (E4, 1 case); required external libraries not provided in the audit package (E5, 2 cases); bugs whose manifestation requires state changes across separate transactions and therefore fall

Table 6: Benchmark dataset composition.

Source	Error Pattern	Count
NumScout	Div In Path	1
	Exchange Problem	1
	Operator Order Issue	1
	Precision Loss Trend	2
	Minor Amount Retention	1
	Profit Opportunity	1
Flyinointment	Greedy Contract	1
Web3Bugs	Erroneous Accounting	72
	Inconsistent State Updates	9
Total (collected)		89
Excluded (see below)		-14
Effective		75

outside our single-transaction scope (E6, 3 cases); a case whose bug actually resides in a different contract than the documented target (1 case); and one NumScout case whose “bug” is the inherent truncation of integer division for which no alternative correct implementation exists (E7, 1 case). The remaining **75 instances** form the effective evaluation set used in RQ1–RQ3.

Methodology. For each of the 75 effective instances, we (1) analyse the audit report to understand the root cause, (2) write an intent annotation that captures the expected numeric behaviour described in the report, and (3) run INTENTCHECKER to record whether it reports *Satisfied*, *Violated*, or *Warning*. Cases that produce a non-*Satisfied* verdict are reported as *mitigated* in RQ1; cases for which no annotation can yield such a verdict are classified into the limitation taxonomy (§5.2) and analyzed for annotation expressibility in RQ2.

5.2 RQ1: Mitigation

Setup. For each of the 75 effective benchmark instances (§5.1), we (1) read the audit report and identify the buggy line(s), (2) write @During and/or @Post intent annotations that capture the expected numeric behavior described in the report, (3) supply the debug annotations (@StateVar, @LocalVar, @GlobalVar, and @IReturn) that the function needs to be analyzable under representative inputs, and (4) run INTENTCHECKER on the resulting annotated source. We record the verdict produced by the validation engine—*Satisfied*, *Warning*, or *Violated*—together with the risk score and violated region. For instances where no annotation can yield a non-*Satisfied* signal, we further classify the structural reason according to the limitation taxonomy described below.

Outcomes. Of the 75 effective instances, INTENTCHECKER produces a *Violated* verdict on **20 cases** when supplied with annotations derived from the audit report. The remaining 55 instances are classified as *not_detectable*: INTENTCHECKER cannot, given the structure of its analysis or its annotation model, surface a non-*Satisfied* signal for

Table 7: Intent clause forms used in the 20 mitigated cases. Counts are per case (one annotation per case unless otherwise noted; one case uses two annotations of the same form on adjacent statements).

Annotation type	Clause form	Count
<i>@During (12 cases)</i>		
	Relational comparison (<code>var relOp expr</code>)	5
	Before/After (<code>var(Before relOp After)</code>)	2
	Function-arg check (<code>func.arg[N] relOp expr</code>)	3
	Implication form	1
	Feasibility check (<code>require feasible</code>)	1
<i>@Post (8 cases)</i>		
	Relational comparison (<code>var relOp expr</code>)	4
	Entry/Exit (<code>var(Entry relOp Exit)</code>)	2
	Return expression (<code>returnExpression relOp expr</code>)	1
	<code>changed(var, true/false)</code>	1
Total		20

them. We first describe the 20 mitigated cases in detail (pattern coverage, annotation usage, and analysis time) and then present the limitation taxonomy for the 55 unmitigated cases.

Pattern coverage of the 20 mitigated cases. The 20 mitigated instances cover 7 of the 8 single-transaction error patterns enumerated in §2.1: Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, Minor Amount Retention, Erroneous Accounting (with both Wrong Interest Rate Order and Wrong Checkpoint Order sub-patterns appearing), Inconsistent State Updates, and Greedy Contract. Profit Opportunity is present in the benchmark but its representative case falls into the L4 category and is not mitigated. This pattern-level distribution confirms that the mitigated cases are not concentrated in a single “easy” pattern but span the breadth of the taxonomy, suggesting that intent-based validation is a general technique rather than a coincidental match for one error class.

Annotation model usage on the 20 mitigated cases. Table 7 reports which intent clauses were used to surface the bug in each mitigated case. Of the 20 cases, 12 use a `@During` clause (mid-execution checks) and 8 use a `@Post` clause (post-execution checks). The single most common form is the simple relational comparison (`intentValue relOp intentValue`), used in 9 cases across both `@During` and `@Post`. The two specialized `@During` forms—`Before/After` comparisons against an injection point, and `func.arg[N]` checks on argument values—are each used 2–3 times, and the specialized `@Post` forms (`Entry/Exit`, `returnExpression`, `changed()`) account for the remainder. This distribution suggests that the bulk of mitigation work in our benchmark is carried by the basic relational form, with the specialized clauses contributing on patterns that the basic form cannot capture (e.g., monotonicity over function execution, argument-level constraints, post-execution change detection).

Table 8: Per-case analysis time on the 20 mitigated cases (mean $\pm$ stdev over 3 runs, single thread, dependency pre-analysis excluded). Cases sorted by mean time.

Case	Mean (s)	Stdev (s)	Range (s)
div_in_path_WANGMI	14.94	2.56	12.0–16.8
web3bugs_45_H.01	13.55	4.52	9.4–18.4
web3bugs_58_H.02	9.67	3.66	7.0–13.8
indivisible_amount_BoostToken	5.67	0.60	5.0–6.1
operator_order_issue_BoostToken	5.43	0.48	4.9–5.8
web3bugs_62_H.08	4.76	0.32	4.4–5.1
web3bugs_70_H.10	3.32	0.31	3.0–3.6
web3bugs_51_H.02	3.18	0.69	2.8–4.0
web3bugs_56_H.02	2.37	1.45	1.5–4.0
profit_opportunity_HIT	2.03	0.46	1.6–2.5
web3bugs_5_H.12	1.75	1.41	0.9–3.4
web3bugs_47_H.02	1.42	0.11	1.3–1.5
web3bugs_5_H.08	1.10	0.17	1.0–1.3
web3bugs_60_H.01	1.06	0.07	1.0–1.1
web3bugs_101_H.01	0.87	0.04	0.8–0.9
exchange_problem_Nokon	0.67	0.09	0.6–0.8
greedy_contract_SwordCrowdsale	0.66	0.07	0.6–0.7
web3bugs_78_H.02	0.58	0.02	0.6–0.6
web3bugs_5_H.07	0.44	0.12	0.3–0.6
web3bugs_77_H.01	0.33	0.02	0.3–0.4
Mean of means	3.69	—	—

Analysis time and what drives it. We measured per-case end-to-end analysis time over three independent runs of each mitigated case on a single thread, excluding the one-time dependency pre-analysis phase (consistent with the development-time scenario, in which inherited contracts and libraries are cached across editing sessions). The mean per-case time is **3.69 s**, the minimum is 0.33 s (**web3bugs_77_H.01**), and the maximum is 14.94 s (**div_in_path_WANGMI**); 15 of the 20 cases finish in under 5 s, and all but two finish in under 10 s. Per-case standard deviations are small for short cases (under 0.15 s) and larger for the long-running cases (e.g., **web3bugs_45_H.01** at 13.5 ± 4.5 s), reflecting OS and JIT noise that becomes proportionally smaller as case complexity grows. The full per-case timings appear in Table 8.

To understand what drives the variation across cases, we collected six per-case complexity metrics—total source lines in the deployment unit, function count, control-flow branch count, loop count, debug annotation count, and external call count—and computed the Pearson correlation of each with the mean analysis time. Figure 4 plots each metric against analysis time. Three observations emerge. *First*, the strongest predictors are aggregate size measures: source lines ($r = 0.87$), function count ($r = 0.76$), and branch count ($r = 0.72$). The dominant cost is therefore the abstract interpreter’s traversal of the deployment unit’s control-flow graph, not any single feature local to the bug site. *Second*, debug annotation count is also strongly correlated ($r = 0.80$),

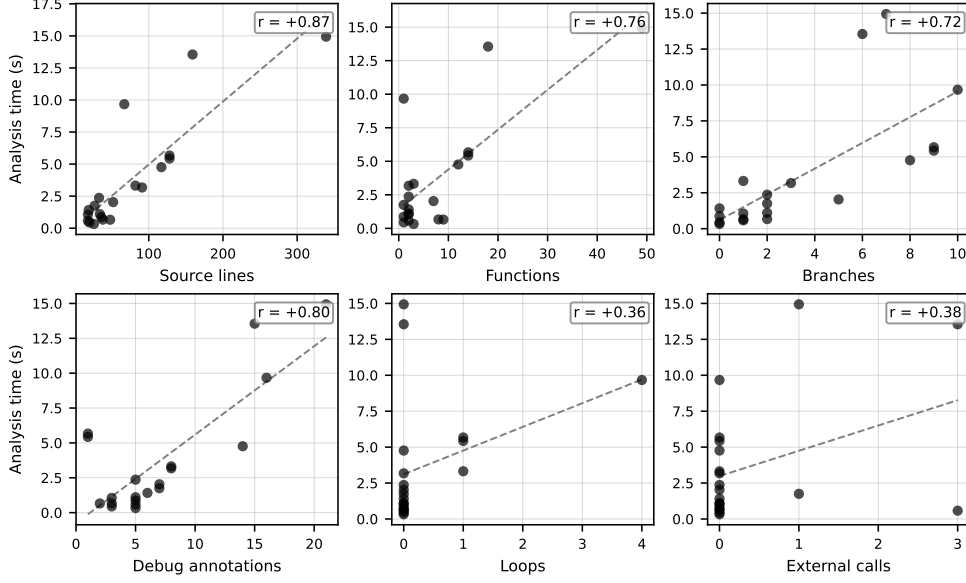


Fig. 4: Scatter plots of per-case analysis time against six complexity metrics (20 mitigated cases, 3-run mean). Dashed lines show linear regression; r is the Pearson correlation coefficient.

reflecting that each debug annotation injects an additional state-evaluation point into the interpreter and thus contributes proportionally to total work. *Third*, and perhaps counter-intuitively, loop count is only *moderately* correlated ($r = 0.36$), and external call count only weakly so ($r = 0.38$). This contradicts the natural expectation that fixpoint iteration on loops would dominate analysis time. Inspection of the mitigated set explains the gap: in several cases that contain loops, the widening of accumulator variables happens to occur on variables that do not flow into the intent annotation’s value path, so the precision loss does not affect the verdict but the fixpoint iteration cost is still paid. The cost is real, but it is not concentrated in the cases that succeed—those cases instead pay it as “dead work” alongside successful bug surfacing. A future optimization that confines fixpoint iteration to the intent-relevant scope could reduce this overhead without changing detectability; we discuss this further in §6.2.

Per-tool latency comparison against NumScout and GPTScan is reported in §5.4.

Limitation taxonomy for the 55 unmitigated cases. The structural reasons for the 55 *not_detectable* instances fall into eleven sub-categories grouped into two top-level families, summarized in Figure 5.

Limitation type definitions.

- **L1a (Loop widening, 8).** Accumulator variables ($+=$, $*=$) inside fixpoint iteration are widened to $\top$, and both buggy and correct executions land in the same widened range.

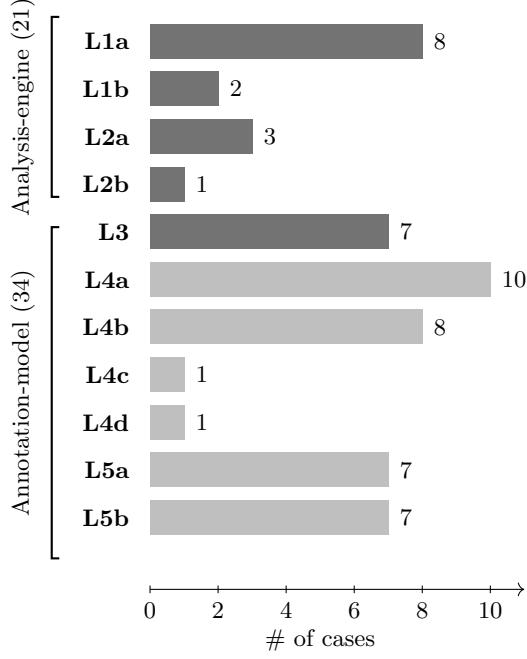


Fig. 5: Limitation breakdown for the 55 *not_detectable* instances. Analysis-engine limitations (L1–L3, 21 cases) are precision losses internal to the abstract interpreter that no developer annotation can recover. Annotation-model limitations (L4–L5, 34 cases) are cases the interpreter *would* distinguish given the right annotation, but the current model cannot express that annotation (L4) or only with bug awareness (L5).

- **L1b (Loop body granularity, 2).** The bug is exposed only at a specific point inside a loop body, but the analyzer summarizes loops by fixpoint result rather than by per-iteration trace, so no annotation point is available at the granularity needed.
- **L2a (Interface call return $\top$, 3).** The callee is reached only through an interface variable, the implementation is therefore unavailable, and `@IReturn` is inapplicable because the bug requires the return value of a state-modifying call.
- **L2b (External call, unknown external state, 1).** The implementation exists via dependency pre-analysis, but the callee’s runtime storage state lies outside the deployment unit and the return value collapses to $\top$.
- **L3 (Unsupported construct, 7).** The bug-relevant value flows through an unsupported construct—`abi.decode`, opaque hash builtins, `address(this).balance`, or non-arithmetic Yul—which the analyzer treats as $\top$.
- **L4a (Inexpressible expected value, 10).** The intent annotation grammar can refer to in-scope arithmetic over existing variables, but the correct expected value depends on a new intermediate computation, an external contract’s state, or a function call that does not appear in the program; no `intentValue` expression can be constructed.

- **L4b (No target storage variable, 8).** The buggy function (a view function, a pure function, a thin wrapper around external calls, or a library helper) does not modify any storage variable in the target contract, so no `@Post` clause has anything to attach to.
- **L4c (Magnitude-only difference, 1).** Buggy and correct code change the same state variable in the same direction; only the magnitude differs. `changed()` and `Entry op Exit` both succeed in both versions, and `INTENTCHECKER`’s `PostEntryExit` clause does not currently admit arithmetic relations between `Entry` and `Exit`.
- **L4d (Invariant masked, 1).** A different statement in the same function already modifies the target state variable, so `changed()` is satisfied in both versions. The actual invariant is a multi-variable arithmetic relation (e.g., a product invariant) which `PostEntryExit` cannot express.
- **L5a (Bug-awareness-required: missing code, 7).** A required statement (a function call, a state update) is omitted; the post-condition that would catch this is expressible, but knowing what is missing already presupposes knowing the bug.
- **L5b (Bug-awareness-required: wrong code, 7).** A required statement is present but uses the wrong identifier, operator, or struct field; the corrective annotation is expressible, but writing it requires knowledge of the correct form—and possessing that knowledge would have prevented the bug in the first place.

Reading the limitation breakdown. Two top-level families emerge. The L1–L3 limitations (21 instances combined) are properties of the underlying abstract interpreter: the analyzer assigns $\top$ to a value that should have been concrete enough to distinguish buggy from correct execution, and no developer annotation under the current model can recover that precision. The L4–L5 limitations (34 instances) are properties of the annotation model rather than of the analyzer: the abstract interpreter *would* distinguish the buggy and correct executions if the developer could express the right intent, but the current grammar either has no expression the developer could write (L4) or has one whose construction presupposes already knowing the bug (L5). RQ2 examines this second family in detail, classifying each of the 34 cases by whether the annotation model can express any awareness signal at all—and if so, how directly.

5.3 RQ2: Annotation Model Expressibility

Setup. RQ1 partitioned the 55 *not_detectable* cases into two top-level families: 21 cases that fail because the abstract interpreter loses precision (L1–L3, analysis-engine limitations) and 34 cases that fail because the annotation model cannot express the right intent (L4–L5). RQ2 examines the second family in detail. The question is no longer “did mitigation succeed?”—it did not, by construction—but rather: *if a developer were already aware of the bug, could they at least express an annotation that the validation engine would surface as Violated or Warning?* This separates the *model’s expressibility ceiling* from *practical detectability*, which depends on whether a developer would write that annotation in the first place.

Three-tier classification. For each of the 34 L4–L5 cases we attempted to construct an annotation, assuming full bug awareness, that would yield a non-*Satisfied* verdict

on the buggy code. Each case is then placed into one of three tiers based on the strength of the resulting signal.

- **Inexpressible (L4).** No annotation under the current grammar can yield a non-*Satisfied* verdict, even with full bug awareness. Either the correct value cannot be constructed from in-scope variables (L4a), the buggy function modifies no target storage (L4b), the difference is in magnitude only along the same direction (L4c), or another statement in the same function masks the change (L4d).
- **Expressible-Strong (L5, “Direct”).** The annotation expresses the correct value or relation directly (a `commonClause` of the form `intentValue relOp intentValue`). The buggy version produces a clean *Violated*; the corrected version produces *Satisfied*. The annotation is essentially a fix-equivalent specification.
- **Expressible-Weak (L5, “Indirect”).** The annotation captures only that some state variable changed—or did not change—in a particular direction (`changed(var, true/false)` or `var(Entry relOp Exit)`). The buggy version produces *Violated* because the change does not occur in the expected way, but the annotation does not encode the correct value, only its presence or polarity. We call this an *awareness signal*: it is enough to alert the developer that something is wrong, even though it does not tell them what the correct value is.

Results. The three-tier classification of the 34 L4–L5 cases is summarized in Table 9. Inexpressible cases account for 20 of 34 (59%); Strong and Weak each account for 7 (21% each). Within the L5 group alone, all 14 cases are expressible—14 of 14, evenly split between Strong (7) and Weak (7)—which means the annotation model can encode *some* awareness signal for every case where bug awareness is the only obstacle. The Inexpressible tier maps exactly to the L4 sub-categories (L4a 10, L4b 8, L4c 1, L4d 1), confirming that L4 represents a hard expressibility ceiling rather than a question of developer awareness.

Representative examples. We illustrate one case from each tier to show the qualitative difference in signal.

Inexpressible (L4a) — `web3bugs_51_H.04 (SwapUtils.getYC)`. The bug is that when a swap crosses a target price, the `StableSwap` formula switches its amplifier coefficient mid-trade; the correct behavior is to split the swap at the crossing point and apply each amplifier to its sub-swap. Constructing this requires computing a split point dx_1 that does not appear anywhere in the program and that is not expressible as a closed-form arithmetic combination of in-scope variables (it is the solution to a Newton-method equation). No `intentValue` expression can capture the correct `getY` return, so the annotation grammar’s `intentValue == arithExpr` form is unusable here.

Expressible-Strong (L5b) — `web3bugs_113_H.05 (NFTPairWithOracle.lend)`. The buggy line is `require(params.ltvBPS >= accepted.ltvBPS, ...)`; the correct comparison is `<=` (a lender wants the borrower’s LTV to be at most their accepted ceiling). Under bug awareness, the annotation is direct:

```
// @During params.ltvBPS <= accepted.ltvBPS
```

Table 9: RQ2 expressibility classification of the 34 annotation-model limitation cases (L4–L5). *Inexpressible*: no annotation yields a non-*Satisfied* verdict even with full bug awareness. *Strong*: the annotation encodes the correct value directly. *Weak*: the annotation provides an awareness signal (change detection) without encoding the correct value.

Tier / Sub-category	Count	%
Inexpressible (L4)	20	59
L4a: inexpressible expected value	10	
L4b: no target storage variable	8	
L4c: magnitude-only difference	1	
L4d: invariant masked	1	
Expressible-Strong (Direct)	7	21
L5a (missing code)	2	
L5b (wrong code)	5	
Expressible-Weak (Indirect)	7	21
L5a (missing code)	5	
L5b (wrong code)	2	
Total	34	100

The buggy code admits inputs with `params.ltvBPS > accepted.ltvBPS`, which violate the annotation cleanly. The annotation is essentially the corrective `require`. *Expressible-Weak (L5a)* — `web3bugs_192_H_01` (`Lock.extendLock`). The function performs `transferFrom` for an additional `_amount` but forgets to update the internal accounting variable `totalLocked[_asset]`. Under bug awareness, the developer can write

```
// @Post totalLocked[_asset](Entry < Exit)
```

which fires *Violated* on the buggy version (the variable does not change, so `Entry == Exit`) and is satisfied on a corrected version that adds `totalLocked[_asset] += _amount`. The annotation tells the developer that the variable was supposed to grow, but it does not tell them *by how much*—the precise increment `_amount` is not present in the annotation. This is an awareness signal, not a fix-equivalent specification, and our grammar does not currently support `intentValue` expressions over the `Entry` value, which would be needed to encode the increment exactly.

Implications. Two findings stand out. First, the distinction between L4 and L5 is meaningful: L4’s 20 cases are a hard ceiling that no amount of developer awareness can lift, whereas L5’s 14 cases are a soft ceiling that the annotation model already covers, contingent on whether the developer is aware. Second, within L5, the 50/50 split between Strong and Weak suggests that for half of the awareness-conditional cases, the developer’s annotation would express the actual fix, and for the other half it would express a coarser “something must change here” signal that nevertheless

Table 10: Signal characterization of the four compared tools.

	INTENTCHECKER	ScType	GPTScan	NumScout
Approach	Abstract interp. + Z3	Type inference	LLM + static	Symbolic exec.
Detection model	Specification-driven	Type-driven	Pattern-driven	Pattern-driven
Annotation	During/Post conds.	Financial types	None	None
Target class	Developer-specified	Type inconsistency	DeFi logic (10 rules)	Numeric (5 patterns)

suffices to surface the bug. Future grammar extensions that would enlarge the Strong tier and shrink Inexpressible—specifically, supporting (a) arithmetic expressions over `Entry` values in `PostEntryExit`, which would directly absorb several Weak cases by letting the developer write the exact increment, and (b) a `returnExpression` clause for view functions, which would absorb several L4b cases by giving view-only functions a meaningful annotation target—are concrete directions identified by this analysis and discussed in §6.2.

5.4 RQ3: Comparison with Existing Detection Tools

We compare INTENTCHECKER against three state-of-the-art tools on the 20 mitigated cases from RQ1: **ScType** (ICSE ’24), a static type-checker that assigns financial types (token unit, scaling factor) to variables and detects type inconsistencies; **GPTScan** (ICSE ’24), an LLM-based multi-step pipeline that matches functions against ten DeFi-specific vulnerability rules (price manipulation, slippage, etc.); and **NumScout**, a symbolic-execution engine that searches for five predefined numeric defect patterns (division-in-path, operator order, precision loss, exchange problem, profit opportunity). Each tool was executed three times on the same machine; ScType and NumScout produce deterministic output, while GPTScan’s LLM-based pipeline may vary across runs.

Detection model. A key distinction among these tools is *how* the set of detectable bugs is defined. GPTScan and NumScout operate on a *pattern-driven* model: each tool ships with a fixed catalog of vulnerability patterns (10 rules and 5 patterns, respectively), and only bugs matching those patterns can be detected. When a new class of numeric error emerges, the catalog must be updated by the tool developers. ScType is *type-driven*: it checks whether arithmetic operations preserve financial type consistency, which is more general than a fixed pattern catalog but still limited to errors expressible as type mismatches. INTENTCHECKER is *specification-driven*: developers write `@During/@Post` conditions describing intended behavior, and the tool verifies the implementation against those conditions via abstract interpretation. This allows INTENTCHECKER to target any numeric property expressible within its annotation grammar—including novel error patterns not yet cataloged by existing tools—but requires developer participation. The grammar’s expressibility ceiling was quantified in RQ2.

Table 11: Mitigation results on the 20 RQ1-mitigated cases. ✓ = mitigated (tool produced an actionable signal for the target bug), ✗ = not mitigated (tool was applicable but produced no signal), N/A = out of scope (no applicable rule or type file, or execution infeasible). GPTScan’s rule set does not target numeric logic errors, hence all cases are out of scope.

Case	Contract.Function	IC	ScType	GPTScan	NumScout
WANGMI	WANGMI._transfer	✓	N/A	N/A	✓
Nokon	Nokon.buy	✓	N/A	N/A	✓
SwordCrowdsale	SwordCrowdsale.refundMoney	✓	N/A	N/A	✗
BoostToken (op)	BoostToken.sendETHToTeam	✓	N/A	N/A	✗
BoostToken (ind)	BoostToken.sendETHToTeam	✓	N/A	N/A	✗
HIT	HIT.getTokens	✓	N/A	N/A	✓
5_H_07	Utils.calcAsymmetricShare	✓	✓	N/A	✓
5_H_08	Utils.calcLiquidityUnits	✓	✓	N/A	✓
5_H_12	Pools.getAddedAmount	✓	N/A	N/A	✗
45_H_01	UToken.borrow	✓	N/A	N/A	✓
47_H_02	WrappedIbbtcEth.transferFrom	✓	✓	N/A	✗
51_H_02	SwapUtils.rampTargetPrice	✓	N/A	N/A	N/A
56_H_02	CDP.update	✓	✗	N/A	✗
58_H_02	LpIssuer._chargeFees	✓	N/A	N/A	N/A
60_H_01	OptimisticLedgerLib.settleAccount	✓	✓	N/A	N/A
62_H_08	Stream.updateStreamInternal	✓	N/A	N/A	N/A
70_H_10	LiquidityBasedTWAP.syncVaderPrice	✓	✗	N/A	✗
77_H_01	MathLib.calcLiqTokenQty	✓	N/A	N/A	N/A
78_H_02	RebaseProxy.mint	✓	N/A	N/A	✗
101_H_01	LenderPool._calcPrincipalWithdrawable	✓	✗	N/A	N/A

Finding 1: coverage landscape. Table 11 shows the per-case mitigation results. GPTScan reported findings on 11 of the 20 target files; however, every finding belongs to a different vulnerability class (price manipulation, slippage limit, etc.) than the actual numeric logic error. GPTScan’s rule set contains no rule for erroneous accounting or incorrect arithmetic formulas; consequently, the entire class of numeric logic errors falls outside its scope. This is not a deficiency of GPTScan—the tool was designed for a different threat model—but it demonstrates that a *coverage gap* exists: the bug class targeted by INTENTCHECKER is not addressed by the most recent LLM-based smart-contract analysis tool.

NumScout mitigated the target bug in 5 of 14 executable cases (6 cases could not be evaluated due to Hardhat flatten limitations with cyclic dependencies). Its five predefined patterns partially overlap with our benchmark: division-in-path and exchange-problem patterns matched several cases, but operator-order and precision-loss bugs in Web3Bugs projects were largely not mitigated.

Finding 2: complementarity with ScType. ScType provides the most direct comparison, as both tools target accounting errors and both require manual annotations (financial types for ScType, during/post conditions for INTENTCHECKER). Among the seven overlapping cases, ScType mitigated four: `calcAsymmetricShare`, `calcLiquidityUnits` (both in Vader Protocol’s `Utils.sol`), `transferFrom` (`WrappedIbbtcEth`), and `settleAccount` (`OptimisticLedger`). INTENTCHECKER

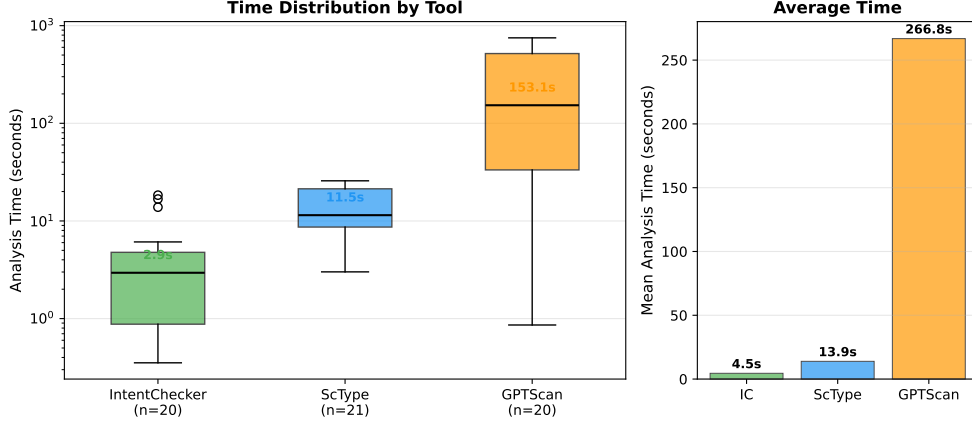


Fig. 6: Analysis time comparison across three tools (log scale). Left: box plot of per-case times over three runs. Right: mean analysis time. INTENTCHECKER and ScType complete in seconds; GPTScan’s LLM pipeline requires minutes per case.

mitigated all seven. The three cases that ScType could not mitigate (`CDP.update`, `syncVaderPrice`, `_calcPrincipalWithdrawable`) are not type-inconsistency errors but rather *value-level* errors: the arithmetic uses correctly-typed operands but produces a numerically wrong result. ScType’s type-driven model cannot express the property “this subtraction should yield the correct fee amount”; INTENTCHECKER’s specification-driven model can. Conversely, ScType’s type inference operates without requiring the developer to know the expected output value, which makes it applicable even when the developer does not have a specific numeric property in mind. The two approaches are thus complementary: ScType catches type-level mismatches automatically, while INTENTCHECKER catches value-level deviations when the developer can articulate the intended behavior.

Finding 3: analysis latency. Figure 6 reports per-case analysis times averaged over three runs. INTENTCHECKER (mean 4.5s) and ScType (mean 13.9s) are both local, deterministic analyses that complete in seconds. GPTScan (mean 267s) is approximately 59× slower due to its multi-step LLM pipeline, and NumScout (mean ~12 minutes for completed cases) requires even more time for its symbolic-execution backend. The sub-10-second latency of INTENTCHECKER makes it suitable for integration into CI/CD pipelines or interactive editor feedback loops, where developers expect near-instant responses.

Summary. Each tool occupies a distinct position in the detection landscape: pattern-driven tools (GPTScan, NumScout) offer fully automatic analysis but can only mitigate bugs within their predefined vulnerability catalogs; type-driven analysis (ScType) generalizes beyond fixed patterns but is limited to errors expressible as type mismatches; and specification-driven analysis (INTENTCHECKER) can mitigate any property within its annotation grammar but requires developer participation. The 20-case evaluation demonstrates that numeric logic errors—a class responsible for some

of the highest-impact DeFi exploits—fall entirely outside the scope of pattern-driven tools and partially outside the reach of type-driven analysis, confirming the practical need for a specification-driven complement.

5.5 Threats to Validity

Internal validity. The intent and debug annotations used in RQ1 were authored by the researchers based on the audit report descriptions, not by the original contract developers. A different annotator could plausibly write a slightly different annotation for the same bug, and a sufficiently weak annotation could fail to surface a bug that a stronger annotation would catch. We mitigate this by deriving each annotation directly from the audit report’s stated root cause and recording the exact annotation alongside each case in the supplementary material, allowing independent re-evaluation.

External validity. Our 89-case benchmark is drawn from Web3Bugs (Code4rena audit findings) and NumScout (BNB Smart Chain), which together cover a substantial portion of publicly known numeric logic errors but are not exhaustive. Findings may not generalize to numeric logic errors that arise in domains underrepresented by these two sources, such as zero-knowledge circuits or non-DeFi applications.

Construct validity. RQ1’s notion of “mitigation” relies on the validation engine producing a non-*Satisfied* verdict (typically *Violated*) for the annotated bug, rather than on a downstream measurement of developer behavior. A non-*Satisfied* verdict on its own does not guarantee that a developer reading it would correctly diagnose and fix the bug; assessing that downstream effect is an important direction for future work. In RQ3, GPTScan is marked as out of scope (N/A) rather than “not mitigated” because its rule set does not target numeric logic errors; this reflects a scope boundary, not a quality deficiency. The ScType comparison is limited to seven overlapping cases for which type annotation files exist; results on the remaining thirteen cases may differ.

6 Discussion

6.1 Annotation Authoring Burden and Future Automation

INTENTCHECKER’s mitigation results in §5.2 are conditional on the developer supplying intent and debug annotations. This is by design—the entire approach is predicated on the developer making their intent explicit—but it nevertheless constitutes the tool’s most significant practical limitation. Two distinct kinds of effort are involved.

Effort 1: writing intent annotations for unfamiliar code. The intent model assumes that the developer can articulate, in the grammar of `@During/@Post` clauses, what the function is supposed to do. For code the developer wrote themselves a few minutes ago, this is unproblematic: the intent is still in the developer’s head. For code the developer is reviewing, modifying, or debugging without prior context, the intent must first be reconstructed from the surrounding code, comments, and external specifications, and only then can it be encoded as an annotation. This reconstruction step is not free, and it is exactly the step that the L5 (bug-awareness-required) limitation in §5.2 measures: when the correct intent is non-obvious, an annotation that captures it cannot be written without already knowing the bug.

Effort 2: supplying debug annotations for realistic execution paths. Even for code whose intent is clear, INTENTCHECKER can only analyze the path that the developer’s debug annotations actually exercise. `@StateVar`, `@LocalVar`, and `@IReturn` require the developer to choose representative ranges; choosing ranges that miss the buggy region produces a false negative, while choosing ranges that are too wide invites widening and produces uninformative *Warning* verdicts. This is the standard cost of any annotation-based analysis and is well understood, but it remains a non-trivial barrier to adoption.

The circularity objection. A natural question is whether writing a correct intent annotation already presupposes knowing the bug—in which case the tool would only confirm what the developer already knows. The concern is legitimate, and our RQ2 analysis quantifies it precisely: 14 of the 34 annotation-model limitation cases (L5) require bug awareness before the annotation can be written, confirming that for those cases, independent discovery through annotation alone is not realistic. However, this circularity is not unique to INTENTCHECKER; it is inherent in *any* specification-based approach, from formal pre/post-conditions to Scribble annotations to test oracles. What differs across tools is how much of the specification burden falls on the developer. For the 20 mitigated cases in RQ1, the annotations were derivable from the developer’s own stated intent (e.g., “this fee should equal x ”, “this balance should decrease after withdrawal”) without requiring knowledge of the specific bug. The L5 cases, by contrast, identify the boundary where intent-driven annotation reaches its limit and external assistance—such as automated annotation suggestion—becomes necessary. This observation directly motivates the LLM-assisted workflow described below: rather than expecting the developer to write all annotations from scratch, a suggestion layer can propose candidate annotations that the developer accepts, rejects, or refines, shifting the task from *generation* to *validation* and partially breaking the circularity.

Future direction: LLM-assisted annotation suggestion and auto-repair. A natural way to lower both barriers is to combine INTENTCHECKER with a large language model that observes the same source the developer is editing. For intent annotations, the LLM could generate candidate `@During`/`@Post` clauses by reading the surrounding function, the contract’s NatSpec comments, and the developer’s recent commit messages, presenting the candidates as suggestions that the developer accepts or edits. For debug annotations, the LLM could propose representative ranges based on the function’s call sites elsewhere in the codebase or based on standard token decimals (e.g., 18 for most ERC20 tokens, 6 for USDC). Beyond annotation suggestion, an LLM that has access to INTENTCHECKER’s *Violated* verdict and violated region could also propose code edits that move the violated region out of the reachable interval, effectively turning the awareness signal into a candidate fix. We deliberately leave both directions as future work; what matters for this paper is that they are natural extensions enabled by, rather than blocked by, the current design.

Developer study. This paper evaluates INTENTCHECKER through structural analysis—what the tool can and cannot detect (RQ1), what the annotation model can and cannot express (RQ2), and how the tool compares with existing detectors (RQ3)—rather than through a developer study measuring perceived usability or annotation intuitiveness. A controlled study with Solidity-experienced participants would answer

a complementary set of questions: whether developers find the annotation syntax natural, how long annotation authoring takes in practice, and whether the tool’s feedback leads to faster bug discovery compared to existing debugging workflows. Designing such a study requires careful attention to participant recruitment (sufficient Solidity expertise) and ecological validity (realistic development tasks rather than isolated code snippets), and we defer it to future work as an independent evaluation axis that builds on the structural foundation established here.

6.2 Remaining Technical Limitations

In addition to the annotation authoring burden discussed above, three classes of technical limitations remain in the current implementation.

Multi-transaction invariants. INTENTCHECKER’s analysis scope is multi-contract but *single-transaction*. A whole class of important numeric properties cannot be expressed within a single transaction because they relate quantities established by different transactions in the contract’s lifetime. The canonical example is the ERC20 conservation invariant `totalSupply == sum(balances)`: any single function execution may temporarily increment one side and not the other, but across the entire transaction history the two must agree. Another example is the price oracle manipulation pattern, where the buggy behavior only manifests when one transaction sets up state that a later transaction reads. Validating such properties would require tracking state across transactions and aggregating across all balance entries, both of which lie outside the current single-transaction interval-domain analysis. This is a known boundary of the development-time intent validation approach we have taken, and is more naturally addressed by post-hoc tools or formal verification frameworks that analyze entire contract lifecycles.

Non-arithmetic Yul and other unsupported Solidity constructs. The arithmetic-focused Yul subset described in §3.2(d) covers the cases where inline assembly is used as a faster substitute for high-precision integer arithmetic, which is by far the most common use case in DeFi libraries. However, hand-written Yul that uses memory and storage opcodes (`mload/mstore`, `sload/sstore`), control flow (`if`, `for`, `switch`), or function definitions falls outside the supported subset and is conservatively treated as $\top$. The same conservative treatment applies to a small number of Solidity constructs that the abstract interpreter does not model: `abi.decode` from raw bytes, `keccak256`, `address(this).balance` (whose value depends on the EVM-level execution context rather than on tracked state), and other built-ins whose results depend on opaque cryptographic or VM-level operations. Bugs whose buggy/correct distinction depends on the precise value produced by such constructs are correspondingly out of INTENTCHECKER’s reach (these account for the L3 cases in Figure 5). Extending the Yul visitor to handle memory and storage opcodes and the abstract interpreter to handle a wider set of opaque builtins is a meaningful but bounded engineering task.

Soundness bounded by debug annotation scope. INTENTCHECKER’s analysis is sound *within* the ranges declared by debug annotations: when the validation engine reports *Satisfied*, the declared intent provably holds for every combination of values in those ranges. The guarantee does not extend outside those ranges. A bug that

only manifests at the boundary of a parameter’s representable values, but not in the range the developer chose for `@LocalVar`, will not appear in any verdict. This is an inherent characteristic of debugging tools—traditional debuggers also only execute the inputs the developer provides—and we encourage developers to use representative ranges and to iteratively refine them when *Warning* results draw attention to specific sub-intervals.

Value-level intent vs. logic-level intent. A broader observation from the limitation taxonomy is that `INTENTCHECKER`’s annotation model operates at the *value level*: it can express whether a variable’s value satisfies a numeric relationship (“this fee should be at least x ”, “this balance should decrease”), but it cannot express whether the right *computation* was performed in the first place. The 20 mitigated cases in RQ1 are precisely the cases where a value-level assertion suffices to distinguish buggy from correct execution. The L4 cases reveal the boundary: in L4a (10 cases), the correct expected value requires a computation that does not exist in the program; in L4b (8 cases), the function modifies no state variable, so there is no value to assert on. These are not failures of precision but of *abstraction level*—the developer’s true intent is about what the code should *do* (“split the swap at the crossing point”, “update `totalLocked`”), not about what value a variable should hold. Capturing such logic-level intent would require a fundamentally different formalism, such as temporal properties over execution traces or structural specifications of required code patterns, and lies outside the scope of a value-based interval-domain tool. Recognizing this boundary is important: `INTENTCHECKER` is effective when the bug manifests as a wrong value, and complementary tools are needed when the bug manifests as a missing or misplaced operation.

6.3 Practical Annotation Guidelines

The annotation grammar (§4) defines *what* can be written; the question that remains is *when* a developer should write which annotation. The 20 mitigated cases and 14 L5 expressible cases reveal a recurring pattern: the most impactful annotations are not the ones that verify what the developer *did* write, but the ones that catch what the developer *forgot* to write. We distill three non-obvious guidelines from this observation.

Guideline 1: Enumerate all state variables a function *should* affect, not just those it *does* affect. Developers naturally reason about the code they wrote—the variables they explicitly assigned, the calls they explicitly made. Bugs of omission (L5a, 7 cases) arise precisely from this cognitive bias: a state variable that *should* have been updated was simply never mentioned. After writing a function, the developer should list every state variable whose value the function is expected to change and add at least a directional annotation (`@Post var(Entry<Exit)` or `@Post changed(var, true)`) for each. In our data, this single practice—checking what *should* change rather than what *does* change—would have caught 7 of the 14 L5 expressible cases and 5 of the 8 mitigated `@Post` cases, all without requiring any knowledge of the specific bug.

Guideline 2: When two state variables should move together, annotate their relationship, not each one in isolation. Several benchmark cases involve accounting pairs (e.g., a per-user balance and a global total, a deposit ledger and

a reward tracker) that should stay in sync. Annotating each variable independently catches only missing-update bugs; annotating their *relationship* (e.g., `@Post globalTotal(Entry-Exit) >= userBalance(Entry-Exit)`) catches bugs where both variables are updated but by inconsistent amounts. This pattern recurs in 4 of the 20 mitigated cases (all Erroneous Accounting) and is the dominant form in L4d (invariant-masked), where independent annotations on each variable succeed but the cross-variable invariant fails.

Guideline 3: For multi-step arithmetic, annotate the result against the mathematically ideal expression, not the code as written. Solidity’s integer division truncates at every intermediate step, and the resulting precision loss depends on the order of operations. Code that reads `amount.div(12).mul(5)` may look correct, but the developer’s actual intent is the mathematical value $\lfloor \text{amount} \times 5/12 \rfloor$. Writing the intent annotation in the ideal form—`@During result >= amount * 5 / 12`—causes `INTENTCHECKER` to compare the truncated code path against the non-truncated reference, surfacing the discrepancy automatically. This pattern accounts for 7 of the 20 mitigated cases (Div In Path, Operator Order Issue, Exchange Problem, and Precision Loss Trend), all of which were caught by annotations that express the mathematically correct expression rather than restating what the code already does.

7 Related Works

7.1 Numeric Logic Error Detection in Solidity

Logic error detection focuses on identifying discrepancies between developer intentions and actual program behavior in smart contracts.

(Eshghie et al., 2024) addresses business logic violations in smart contracts by using Dynamic Condition Response (DCR) graphs to model expected behaviors including temporal constraints, role-based access control, and inter-function dependencies. The system monitors runtime transactions against user-provided DCR models and their function mappings, generating violation reports when execution activities deviate from the specified business logic patterns.

(Stephens et al., 2021) verifies temporal properties of smart contracts by accepting user-specified Linear Temporal Logic (LTL) formulas and attack models to check both safety and liveness properties. The system not only detects violations but also generates concrete attack scenarios that demonstrate how the temporal properties can be violated, providing developers with actionable insights into potential logic errors.

(Wei et al., 2024) enables developers to specify function and address behaviors using the ConSol syntax alongside their Solidity code, then verifies whether the implementation adheres to these behavioral specifications and generates behavior-compliant Solidity programs without annotations.

(Wang et al., 2024b) leverages multimodal learning to automatically infer invariants and detect machine unauditable bugs (MUBs) (Zhang et al., 2023) by analyzing both source code and natural language artifacts including comments, function signatures, and documented transaction scenarios. The system employs a fine-tuned model

with Tier of Thought (ToT) prompting that progressively predicts transaction contexts, generates and ranks invariants, and ultimately identifies vulnerabilities through three-tier reasoning.

(Zhang, 2024) proposes a novel type system for Solidity variables defined as tuples of (Financial Type, Scaling Factor, Token Unit, Address) to detect accounting errors through type checking. The system reinterprets Solidity code based on detection patterns such as incorrectly adding balances with fees, enabling systematic identification of numeric logic errors that arise from mismatched financial operations.

Furthermore, there are also approaches that address multiple transaction logic errors such as price manipulation (Wu et al., 2023; Wang et al., 2024a; Xi et al., 2024; Arora et al., 2024; Kong et al., 2023) and flash loan attacks (Li et al., 2025; Ramezany et al., 2023).

The numeric logic error detectors above and INTENTCHECKER occupy distinct points along the development-auditing timeline rather than competing on the same axis. NumScout, GPTScan, SmartInv, and the type-system approach of Zhang (2024) are all designed to run over completed code and recover developer intent indirectly through pattern catalogs, language-model inference, or financial type assignments. INTENTCHECKER is positioned earlier in time and lets the developer declare intent explicitly through annotations evaluated inside the editor. Because the two approaches see different inputs (code only vs. code with declared intent) and run in different workflow stages (post-hoc audit vs. edit-time validation), their coverage is necessarily complementary, and §5.4 characterizes that complementarity rather than arguing numerical superiority.

7.2 Specification-based Smart Contract Verification

Specification-based verification approaches focus on verifying general properties and correctness requirements of smart contracts rather than specific developer intentions.

(Consensys Diligence, 2025c) provides a framework that transforms developer annotations into Solidity assertion code, enabling testing with existing tools like MythX (Consensys Diligence, 2025b) and Mythril (Consensys Diligence, 2025a). The framework supports multiple annotation types including `if.succeed` (conditions for successful function returns), `invariant` (contract-level properties that must always hold), `if.updated` (conditions checked when state variables change), and `assert` (validations at specific execution points), making formal specifications accessible through familiar testing pipelines.

(Wesley et al., 2022) builds upon Scribble (Consensys Diligence, 2025c) to verify whether Solidity code satisfies specifications written in the Scribble language, leveraging C-based verification tools like SeaHorn (Gurfinkel et al., 2015), libFuzzer (LLVM, 2025), and KLEE (Cadaru et al., 2008) by transforming smart contracts into LLVM-IR based harnesses. The framework abstracts the 2^{160} possible user addresses into a small set of representative users (implicit, transient, and explicit), enabling verification of properties that must hold for arbitrary numbers of users, such as accounting properties like “the contract balance equals the sum of all user deposits.”

(Nguyen et al., 2023) verifies Solidity code using two types of annotations: specification annotations (loop invariants, call invariants, and call modifies) that developers

declare as assumptions, and verification annotations (ensures, reverts_if, and contract invariants) that the tool checks against the implementation. The system assumes the truth of specification annotations and uses them to verify whether the code satisfies function post-conditions, correctly handles revert conditions, and maintains contract invariants throughout execution.

(Bartoletti et al., 2024) focuses on verifying liquidity properties of smart contracts through formal methods by converting both the contract code and user-defined liquidity properties into SMT constraints. The system then checks whether the contract satisfies these properties using SMT solvers, generating counterexamples when violations are detected to help developers understand potential liquidity issues.

While specification-based approaches focus on verifying high-level contract properties and invariants in completed code, IntentChecker addresses a fundamentally different problem: validating specific developer intentions about numeric values during the debugging process. Unlike existing tools that treat library functions as black boxes, IntentChecker performs interval-domain analysis that tracks actual value ranges through library function calls (e.g., SafeMath operations), enabling developers to verify their step-by-step numeric intentions rather than abstract contract-level properties.

7.3 Tools for Strengthening Security During Development Phase

Development-phase security tools integrate vulnerability detection directly into the coding workflow, enabling developers to identify and address security issues as they write code rather than in post-development analysis. (Ren et al., 2021) presents an integrated development environment for smart contract development that combines natural language model-based code autocompletion with security validation. The platform integrates existing static analysis tools including Oyente (Luu et al., 2016), Mythril (Consensys Diligence, 2025a), and Securify (Tsankov et al., 2018) to provide comprehensive security reports during development workflow.

(Protofire, 2025) provides a Solidity linter that detects predefined vulnerability patterns including low-level calls, reentrancy, and deprecated function usage during development. The tool enforces security best practices by checking for patterns such as proper visibility declarations, send result validation, and secure compiler version usage.

While these development-phase tools rely on predefined detection patterns for known vulnerabilities, INTENTCHECKER enables developers to specify their intended numeric behaviors through annotations and validates whether the actual program execution matches these specified intentions. INTENTCHECKER also extends SOLQDEBUG’s analysis scope beyond a single contract to a multi-contract single-transaction scope that includes inheritance, libraries, interfaces, and an arithmetic Yul subset, which is what makes intent validation feasible on realistic DeFi code.

8 Conclusion

We presented INTENTCHECKER, a development-time intent validation tool that extends SOLQDEBUG with (i) an intent annotation model (`@During/@Post`), (ii) a validation engine producing graded *Satisfied/Warning/Violated* signals with risk scores and violated regions, and (iii) a scope extension to multi-contract single-transaction analysis covering inheritance, libraries, interfaces, and an arithmetic Yul subset.

On 75 effective benchmark instances (89 collected, 14 excluded) from Web3Bugs and NumScout, INTENTCHECKER surfaced 20 bugs via *Violated* verdicts at edit-time speeds (mean 3.69s per case). RQ2’s expressibility analysis showed that 14 of the 34 annotation-model limitation cases are expressible under bug awareness, while 20 represent a hard grammar ceiling—identifying concrete extension points for the annotation model. INTENTCHECKER’s positioning relative to existing detection tools is complementary: it fills the edit-time gap that auditing-time tools structurally cannot occupy.

The primary practical limitation is the annotation authoring burden, which future work can address through LLM-assisted annotation suggestion (lowering the L5 bug-awareness ceiling) and auto-repair from violated regions. On the analyzer side, extending Yul support and multi-transaction reasoning would close the remaining technical limitations. More broadly, this work adds “edit-time intent validator” as a third category alongside pattern-based detectors and formal verifiers in the design space for numeric logic error tools.

References

- Arora, S., et al.: SecPLF: Secure protocols for loanable funds against oracle manipulation attacks. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS), pp. 1394–1405 (2024). <https://doi.org/10.1145/3634737.3637681>
- Bartoletti, M., et al.: Solvent: liquidity verification of smart contracts. In: International Conference on Integrated Formal Methods (iFM), pp. 256–266 (2024). https://doi.org/10.1007/978-3-031-76554-4_14
- Cadar, C., et al.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, pp. 209–224 (2008). <https://doi.org/10.5555/1855741.1855756>
- Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3623302>
- Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). <https://doi.org/10.1109/TSE.2025.3555622>
- Consensys Diligence: Mythril. <https://github.com/ConsenSysDiligence/mythril> (2025). Accessed December 2025
- Consensys Diligence: MythX. <https://mythx.io/> (2025). Accessed December 2025

- Consensys Diligence: Scribble. <https://diligence.security/scribble/> (2025). Accessed January 2026
- Eshghie, M., et al.: Highguard: Cross-chain business logic monitoring of smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 2378–2381 (2024). <https://doi.org/10.1145/3691620.3695356>
- Gurfinkel, A., et al.: The SeaHorn verification framework. In: International Conference on Computer Aided Verification, pp. 343–361 (2015). https://doi.org/10.1007/978-3-319-21690-4_20
- Jeon, I., et al.: SolQDebug: Debug Solidity Quickly for Interactive Immediacy in Smart Contract Development (2025). <https://doi.org/10.21203/rs.3.rs-8077153/v1>
- Kong, Q., et al.: Defitainter: Detecting price manipulation vulnerabilities in defi protocols. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 1144–1156 (2023). <https://doi.org/10.1145/3597926.3598124>
- Li, X., et al.: Penetrating the hostile: Detecting defi protocol exploits through cross-contract analysis. IEEE Transactions on Information Forensics and Security 20, 11759–11774 (2025). <https://doi.org/10.1109/TIFS.2025.3626979>
- LLVM: libFuzzer. <https://llvm.org/docs/LibFuzzer.html> (2025). Accessed December 2025
- Luu, L., et al.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 254–269 (2016). <https://doi.org/10.1145/2976749.2978309>
- Nguyen, T.D., et al.: An idealist’s approach for smart contract correctness. In: International Conference on Formal Engineering Methods, pp. 11–28 (2023). https://doi.org/10.1007/978-981-99-7584-6_2
- Protofire: Solhint. <https://github.com/protofire/solhint> (2025). Accessed January 2026
- Ramezany, S., Setthawong, R., Setthawong, P.: Midnight: An Efficient Event-driven EVM Transaction Security Monitoring Approach For Flash Loan Detection. In: Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE), pp. 237–241 (2023). <https://doi.org/10.1109/JCSSE58229.2023.10201970>
- Ren, M., et al.: Making smart contract development more secure and easier. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp. 1360–1370 (2021). <https://doi.org/10.1145/3468264.3473929>
- Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. Empirical Software Engineering 29(1), 13 (2024). <https://doi.org/10.1007/s10664-023-10398-5>
- Stephens, J., et al.: SmartPulse: automated checking of temporal properties in smart contracts. In: Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP), pp. 555–571 (2021). <https://doi.org/10.1109/SP40001.2021.00085>

- Sun, Y., et al.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3639117>
- Tsankov, P., et al.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 67–82 (2018). <https://doi.org/10.1145/3243734.3243780>
- Wang, D., et al.: Defiguard: A price manipulation detection service in defi using graph neural networks. *IEEE Transactions on Services Computing* (2024). <https://doi.org/10.1109/TSC.2024.3489439>
- Wang, S.J., Pei, K., Yang, J.: Smartinv: Multimodal learning for smart contract invariant inference. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2217–2235 (2024). <https://doi.org/10.1109/SP54263.2024.00126>
- Wei, G., et al.: Consolidating Smart Contracts with Behavioral Contracts. *Proceedings of the ACM on Programming Languages* 8(PLDI), 965–989 (2024). <https://doi.org/10.1145/3656416>
- Wesley, S., et al.: Verifying Solidity smart contracts via communication abstraction in SmartACE. In: International Conference on Verification, Model Checking, and Abstract Interpretation, pp. 425–449 (2022). https://doi.org/10.1007/978-3-030-94583-1_21
- Wu, S., et al.: Defiranger: Detecting defi price manipulation attacks. *IEEE Transactions on Dependable and Secure Computing* 21(4), 4147–4161 (2023). <https://doi.org/10.1109/TDSC.2023.3346888>
- Xi, R., Wang, Z., Pattabiraman, K.: POMABuster: Detecting Price Oracle Manipulation Attacks in Decentralized Finance. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 3923–3942 (2024). <https://doi.org/10.1109/SP54263.2024.00257>
- Zhang, B.: Towards finding accounting errors in smart contracts. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3639128>
- Zhang, W., et al.: Nyx: Detecting exploitable front-running vulnerabilities in smart contracts. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2198–2216 (2024). <https://doi.org/10.1109/SP54263.2024.00146>
- Yang, S., et al.: Hyperion: Unveiling DApp Inconsistencies Using LLM and Dataflow-Guided Symbolic Execution. In: Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2125–2137 (2025). <https://doi.org/10.1109/ICSE55347.2025.00015>
- Zhang, Z., et al.: Demystifying exploitable bugs in smart contracts. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 615–627 (2023). <https://doi.org/10.1109/ICSE48619.2023.00061>
- Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. *IEEE Transactions on Software Engineering* 50(6), 1360–1373 (2024). <https://doi.org/10.1109/TSE.2024.3383422>

Zhou, L., et al.: Sok: Decentralized finance (defi) attacks. In: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461 (2023). <https://doi.org/10.1109/SP46215.2023.10179435>